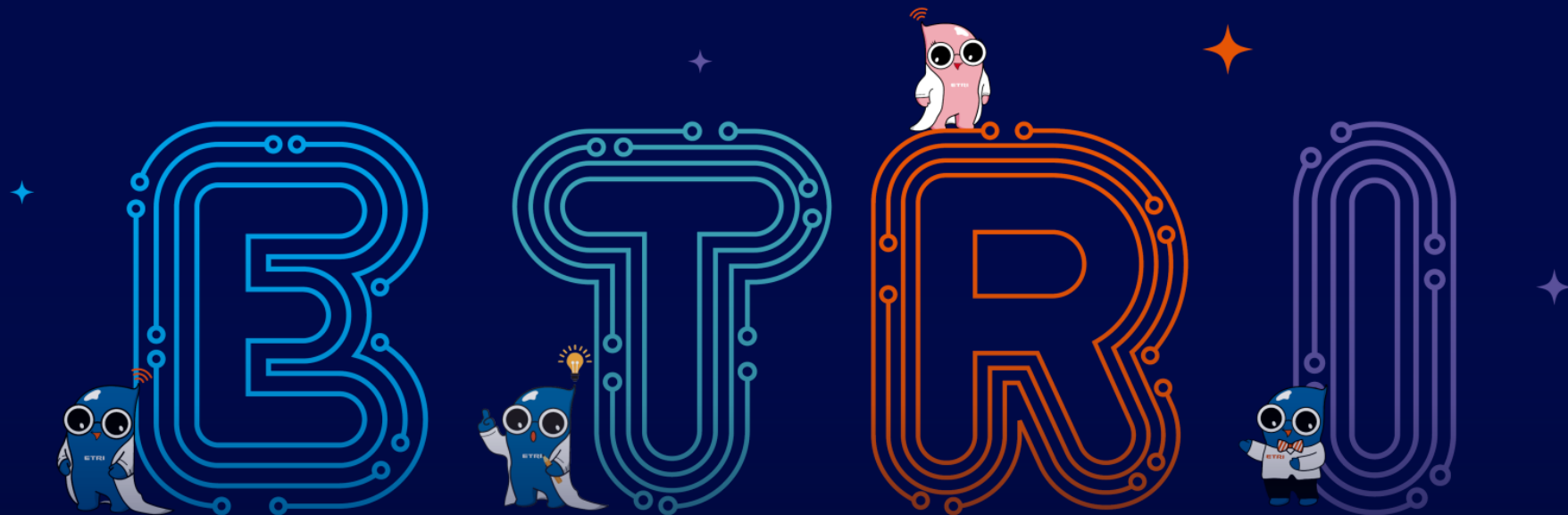




언어모델 기반 자연어처리 실습 기초

1일차 · ChatGPT를 탄생시킨 언어모델 연구 소개 — 시아카데미 [A4021]

2026. 09. 03 ~ 04 류 지 희 (chrisjihee@etri.re.kr) · 한국전자통신연구원

2일 과정 구성과 오늘의 위치

구분	내용	형식	핵심 산출물
1일차 오전	언어모델 연구 개론 (이 자료) — Transformer → BERT → GPT → 최신 상용/오픈 LLM → 핵심 요소기술	강의	연구 흐름의 지도
1일차 오후	BERT 실습 — 주제분류·문장유사도·개체명인식 세 태스크를 각각 미세조정, 같은 세 태스크를 T5로도	실습	태스크별 BERT 모델 3개 + T5 모델
2일차	기계독해 (BERT 4A · T5 4B) → LLM 파인튜닝 실습 — 소형 LLM 하나를 지시문 데이터로 LoRA 학습 → 6개 태스크 동시 수행 → 웹 데모	실습	기계독해 모델 + 내 어댑터 1개 + 데모 페이지
2일차 마무리	BERT · T5 · GPT 계열 — 세 방식의 성능표·구현 방식·자원을 나란히 놓고 비교	토론	비교표와 차트

도입: 요즘 언어모델이 하는 일

◆ 요즘 놀라운 것은 '대답을 잘한다'가 아닙니다

- 화면을 보고 마우스·키보드를 직접 조작해 업무를 끝낸다 — 사람 평균에 근접한 성공률 (GPT-5.4, OSWorld-Verified 75%)
- 몇 시간에 걸쳐 저장소 단위로 코드를 고친다 — 사람이 중간에 지시하지 않아도 (GPT-5.3-Codex)
- 필요한 도구를 스스로 만들어 쓰고, 만든 도구로 다시 문제를 푼다
- 이미지를 새로 그리는 것이 아니라 **대화로 고쳐 나간다** — "배경만 바꿔줘", "이 글자만 지워줘"

◆ 그리고 이런 것도 됩니다 — 오늘·내일 우리가 직접 할 일입니다

- 73억 파라미터짜리 한국어 모델을 **24GB GPU 한 장에서 20여 분** 학습시킨다
- 파라미터의 0.55%만 건드렸는데 여섯 태스크 평균이 **52.0 → 69.7**로 오른다
- 학습한 모델에게 내가 직접 질문하고, 어댑터를 켜고 끄며 전후를 비교한다

◆ 여기서 볼 것

- 질문(지시)만 바꾸면 전혀 다른 일을 한다 — 모델은 하나다
- '생각 중...' 표시: 답하기 전에 스스로 단계별로 추론하는 시간(Test-time Compute)
- 글·그림·코드가 한 대화에서 오간다 — 네이티브 멀티모달

◆ 오늘 강의는 이것들이 어디서 왔는지를 거슬러 올라가는 여행입니다

이 강의를 통해 답하고 싶은 세 가지 질문

◆ ? ChatGPT와 같은 모델은 어떻게 학습되었고, 어떻게 작동하는가?

- Transformer → 사전학습(BERT/GPT) → 지시 미세조정 → 인간 피드백 → 추론 강화, 이 계단을 하나씩 오른다

◆ ? 언어모델은 어떻게 LLM으로 발전하여 AGI 논의의 중심이 되었는가?

- 규모(파라미터·데이터·연산)를 키웠을 때 무엇이 달라졌는가, 그리고 규모 다음에는 무엇이 관건이 되었는가

◆ ? AI 시대의 필수품인 LLM을 앞으로 어떻게 활용해 나가야 할까?

- 성능표 읽는 법, 목적별 도구 선택, 위험과 책임

◆ 🛠 그리고 하나 더: 직접 만들어 본다

- 이론에서 본 지시 미세조정과 LoRA를 2일차에 내 손으로 재현하고, 내가 학습시킨 모델에게 직접 질문해 본다

전체 파트 구성

- 1 Part 1 — 인공지능 및 자연어처리 개요
- 2 Part 2 — 언어모델의 발전 과정 및 원리: Transformer · BERT · T5 · GPT
- 3 Part 3 — 평가와 최신 모델, 그리고 핵심 요소기술
- 4 실습 연결 — BERT·T5·GPT 계열 세 방식의 차이, 2일차 예고
- 5 Part 4 — 변화하는 LLM을 대하는 우리의 자세

PART

I

인공지능 및 자연어처리 개요

- 1 인공지능이란 무엇인가?
- 2 자연어처리란 무엇인가?
- 3 자연어처리의 주요 태스크 — 오늘·내일 실습과의 연결
- 4 딥러닝 기반 자연어처리의 발전



1. 인공지능이란 무엇인가?

◆ 인공지능 (Artificial Intelligence, AI)

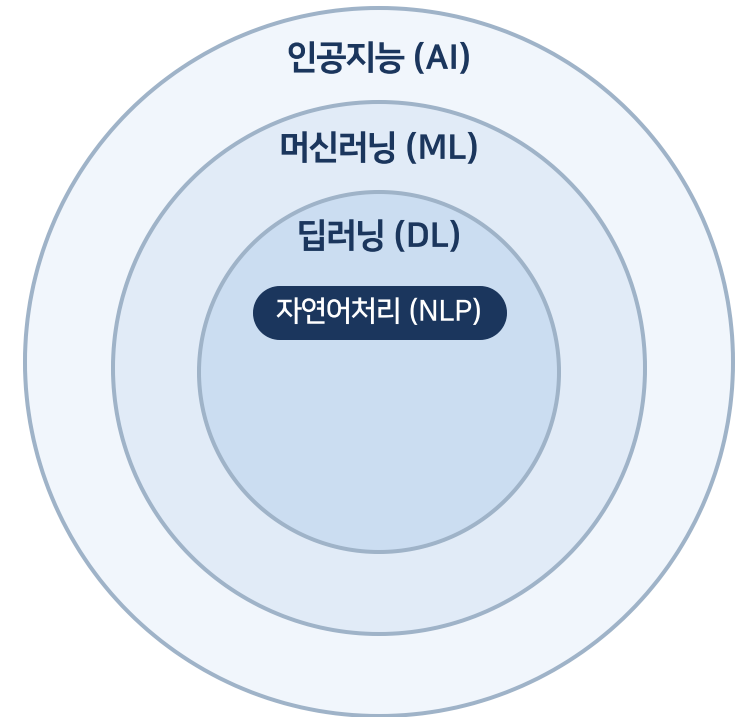
- 인간의 지능을 모방하는 기계 — 컴퓨터 시스템이 인간의 학습, 추론, 인식, 적응 능력을 재현하려는 기술

◆ AI의 주요 구성 요소

- 머신러닝 (Machine Learning): 데이터를 기반으로 학습하고 예측을 수행하는 알고리즘
- 딥러닝 (Deep Learning): 신경망(Neural Networks)을 여러 층으로 쌓아 학습하는 심화된 머신러닝 기법
- 자연어처리 (NLP): 인간의 언어를 이해하고 생성하는 기술 — 오늘의 주제

◆ 한 줄 정리

- 요즘 'AI'라고 부르는 것의 대부분은 '딥러닝으로 만든 언어모델'을 가리킨다



NLP는 언어를 다루는 응용 분야이며, 오늘날 그 방법론의 중심이 딥러닝입니다 (자체 도식)

2. 자연어처리란 무엇인가?

◆ 자연어처리 (Natural Language Processing, NLP)

- 컴퓨터가 인간의 언어를 이해하고 생성할 수 있도록 하는 인공지능의 한 분야
- 텍스트(Written Language)와 음성(Spoken Language)을 처리하여 의미 있는 정보를 추출하고 사람과 상호작용

◆ NLP의 주요 목표

- 언어 이해: 텍스트의 의미를 파악하고 문맥을 이해 — 예) 문서 주제 분류, 질문에 대한 답변
- 언어 생성: 주어진 정보나 문맥을 바탕으로 자연스러운 언어를 생성 — 예) 자동 완성, 기사 작성
- 번역·변환: 한 언어를 다른 언어로 — 예) 영한 번역, 자연어 → 프로그래밍 언어 (Vibe Coding)

◆ 왜 어려운가

- 같은 말이 문맥에 따라 다른 뜻 (다의어), 생략과 대용, 세상 지식이 필요한 추론
- '자연어'는 사람이 쓰기 위해 진화했고, 컴퓨터를 위해 설계된 것이 아니다

3. 자연어처리의 주요 태스크 — 이번 과정에서 직접 다루는 여섯 가지

태스크	입력 → 출력	실습 데이터셋 (한국어, 공개)	평가 지표	BERT 인코더	T5 인코더·디코더	GPT 계열 디코더
주제분류	뉴스 제목 → 7개 분야 중 하나	KLUE-YNAT (연합뉴스 제목)	정확도	○	○	○
문장유사도	문장 쌍 → 0~5점 유사도	KLUE-STs	Pearson 상관	○	○	○
개체명인식	문장 → 인명·지명·기관명... 위치와 유형	KLUE-NER	개체 단위 F1	○	○	○
기계독해	지문 + 질문 → 지문 속 정답 구간	KorQuAD 1.0 (위키백과)	EM	○	○	○
SQL생성	스키마 + 질문 → SQL 한 줄	Spider-ko	문자열 일치	—	○	○
수학추론	문장제 문제 → 풀이와 최종 답	GSM8K-ko	최종 답 일치	—	○	○

앞의 네 태스크는 세 방식을 같은 데이터·같은 평가셋·같은 채점으로 비교한다. 뒤의 두 태스크는 정답이 라벨도 스펠도 아니어서 생성 모델만 가능하다. 수학추론은 시간이 남을 때 다루는 보너스다. **구조의 이름은 인코더·인코더-디코더·디코더이고, 각 자리를 대표하는 모델이 BERT·T5·GPT다.** 요즘 이 디코더 계열을 아주 크게 키운 것을 **LLM**이라 부른다 — LLM은 구조의 이름이 아니다.

왜 수학추론만 보너스인가 — 파인튜닝이 항상 이기지는 않는다

◆ 같은 데이터·같은 방식으로 파인튜닝했는데 결과가 갈렸다

- 다섯 태스크(주제분류·개체명인식·기계독해·문장유사도·SQL생성): **11개 모델 중 11개가 전부 상승** — 태스크별 변화 중앙값 **+21.7 ~ +47.9점**
- 수학추론: 11개 중 **4개만 상승**, 변화 중앙값 **-2.3점** (최소 -20.0, 최대 +35.3) — 내려가는 모델이 더 많다
- 내일 쓰는 `skt/A.X-4.0-Light` 는 **오르는 쪽 넷 중 하나**다 (**51.0 → 53.3**). 같은 데이터·같은 설정인데 EXAONE-4.0-1.2B 는 **44.7 → 32.0** 으로 떨어진다

◆ 왜 그런가

- 풀이 과정을 여러 단계 이어 가야 하는 문제는 **800건짜리 소규모 학습으로는 형식만 따라 하게 되기 쉽다**
- 그 과정에서 모델이 **원래 갖고 있던 추론력을 잃는다** — 답의 모양은 배웠는데 푸는 힘은 줄어든 것

◆ 그래서 이렇게 쓴다

- 정규 실습은 **SQL생성까지 다섯 태스크**. 수학추론은 시간이 남을 때 다루는 **+1 보너스**
- 빠지 않고 남겨 둔 이유가 있다 — '**파인튜닝이 항상 이기지는 않는다**'를 눈으로 보여 주는 사례이기 때문
- 성능표를 읽을 때 평균만 보면 놓치는 것이 무엇인지, 이 한 칸이 알려 준다

4. 딥러닝 기반 자연어처리의 발전

◆ 딥러닝이 중요한 이유

- 다층 신경망으로 데이터에서 직접 표현(feature)을 학습 — 사람이 규칙과 특징을 손으로 설계하지 않는다
- 단어를 숫자 벡터로 (word2vec 등, 2013): $\text{왕} - \text{남자} + \text{여자} \approx \text{여왕}$ 같은 의미 관계가 벡터 연산에 나타남

◆ 주요 신경망 구조

- CNN (합성곱 신경망): 이미지 인식에서 발전, 지역적 패턴(n-gram)을 효과적으로 감지
- RNN / LSTM (순환 신경망): 단어를 순서대로 읽으며 상태를 갱신 — 2017년까지의 표준
- Transformer (2017): Self-Attention으로 모든 단어 쌍의 관계를 한 번에 계산 — 병렬 처리와 긴 문맥 이해

◆ 패러다임의 전환: 사전학습 + 미세조정 (Pre-training + Fine-tuning)

- 대량의 텍스트로 '언어 자체'를 먼저 배우고(사전학습), 적은 데이터로 특정 일을 가르친다(미세조정)
- BERT(2018)가 이 방식을 표준으로 만들었고, 오늘 오후 실습이 정확히 이것이다

PART

II

언어모델의 발전 과정 및 원리

- 1 언어모델의 개요 및 초기 모델
- 2 Transformer의 등장
- 3 BERT의 혁신
- 4 T5의 통합적 접근
- 5 GPT 시리즈의 등장과 발전 — 그리고 규모의 법칙



1. 언어모델(Language Model)의 개요

◆ 언어모델 (LM)

- 단어·문장의 연속적인 패턴을 학습하여 다음에 나올 단어(토큰)를 예측하는 모델
- 수식으로는 $P(\text{다음 토큰} \mid \text{지금까지의 문맥})$ — 확률분포를 출력한다

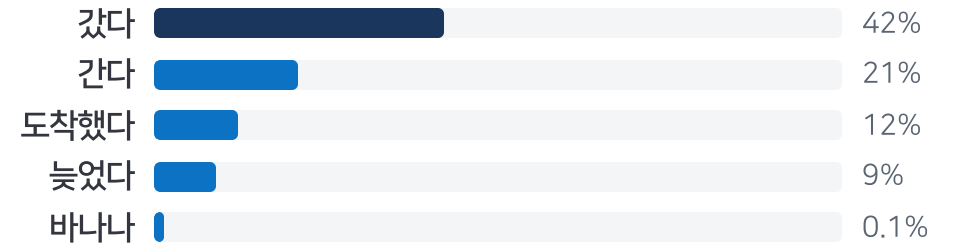
◆ 주요 기능

- 텍스트 예측: 입력된 텍스트의 다음 단어나 문장을 예측 (자동 완성)
- 문맥 이해: 잘 예측하려면 문맥의 의미와 세상 지식을 내부에 담아야 한다
- 문장 생성: 예측한 토큰을 붙이고 다시 예측하는 반복으로 새로운 텍스트를 만든다

◆ 왜 '다음 단어 맞히기'가 지능처럼 보이는가

- 인터넷 규모의 텍스트에서 다음 단어를 잘 맞히려면 문법·사실·논리·상식을 전부 압축해야 한다
- ChatGPT도 본질은 이 함수다 — 다만 규모와 학습 방식이 달라졌다

나는 오늘 아침에 학교에 [?]



언어모델 = 앞 문맥이 주어졌을 때 다음 토큰의 확률분포를
내는 함수. 확률이 높은 것을 고르고, 붙이고, 다시 예측...
이것이 '생성'이다 (자체 도식)

1. 초기 언어모델과 그 한계

◆ 대표적인 초기 언어모델

- N-gram: 앞 $n-1$ 개 단어만 보고 다음 단어의 빈도를 센다 (1990년대~) — 단순하지만 문맥이 매우 짧다
- HMM (은닉 마르코프 모델): 품사 태깅·음성인식에 쓰인 확률 모델
- RNN / LSTM: 신경망이 단어를 순서대로 읽으며 '기억(상태)'을 갱신 (2010년대 표준)

◆ 초기 모델들의 종합적인 한계

- 문맥 이해 부족: 다의어, 문맥에 따른 의미 변화를 반영하지 못해 부자연스러운 텍스트 생성
- 장기 의존성 문제: 문맥이 길어질수록 앞 정보가 희미해져 예측 성능 저하 — 번역·생성에서 문맥 유지가 어렵다
- 병렬 처리의 어려움: 순차 처리라서 학습·추론이 느리고 GPU를 충분히 활용하지 못한다

◆ → 이 세 한계를 한 번에 겨누는 구조가 2017년에 등장한다

2. Transformer의 등장

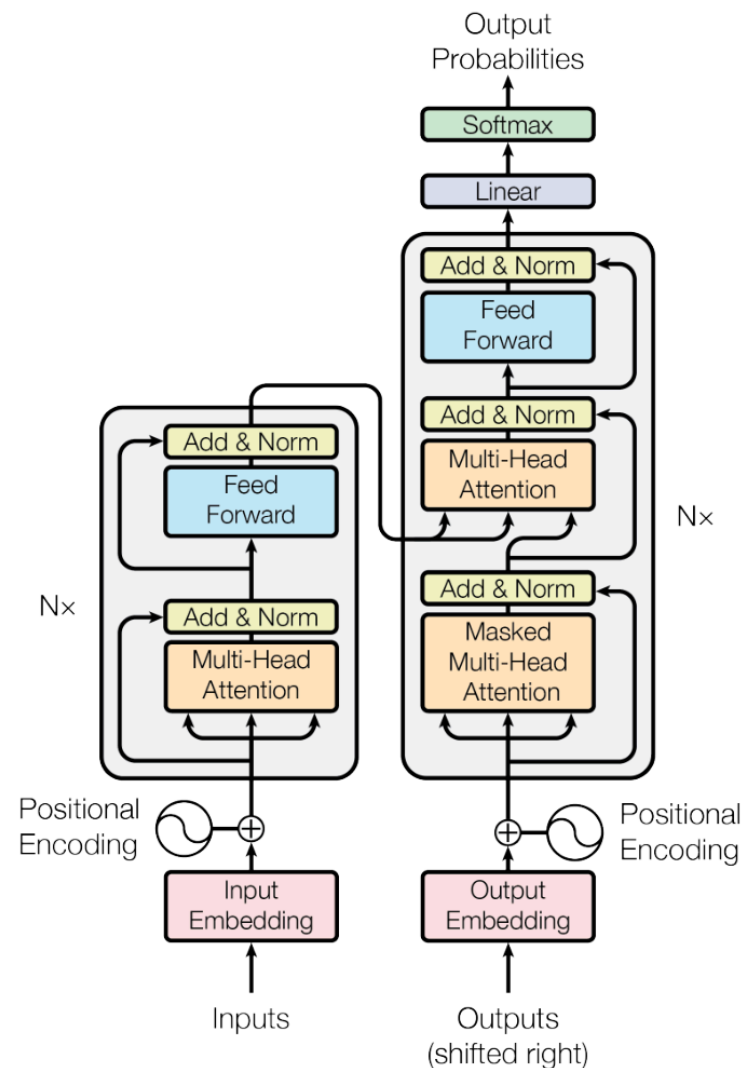
◆ Transformer (Google, 2017)

- Self-Attention 메커니즘을 기반으로 제안된 시퀀스 변환 모델
- RNN이나 CNN을 쓰지 않고도 최고 성능 — 논문 제목이 곧 주장: "Attention Is All You Need"

◆ 아키텍처 구성: Encoder-Decoder

- Encoder (그림 왼쪽): 입력 시퀀스를 받아 Self-Attention으로 각 단어의 문맥 표현을 만든다
- Decoder (그림 오른쪽): Encoder 출력과 지금까지 생성한 단어를 보고 다음 단어를 생성한다
- 두 부분 모두 같은 블록(Attention + Feed-Forward)을 N번 쌓은 구조 — 쌓을수록 커진다

◆ 원래 용도는 기계번역(영→독, 영→불)이었으나, 이 블록이 모든 LLM의 부품이 된다



Transformer 구조 (Figure 1) · 출처: [1] Vaswani et al., 2017

2. Self-Attention의 작동 방식

◆ 기본 개념

- 입력 시퀀스의 각 요소가 다른 모든 요소와의 관계를 통해 자신을 재표현하는 메커니즘
- 모든 단어가 문장의 다른 모든 단어를 고려할 수 있어 문맥 이해력이 뛰어나다

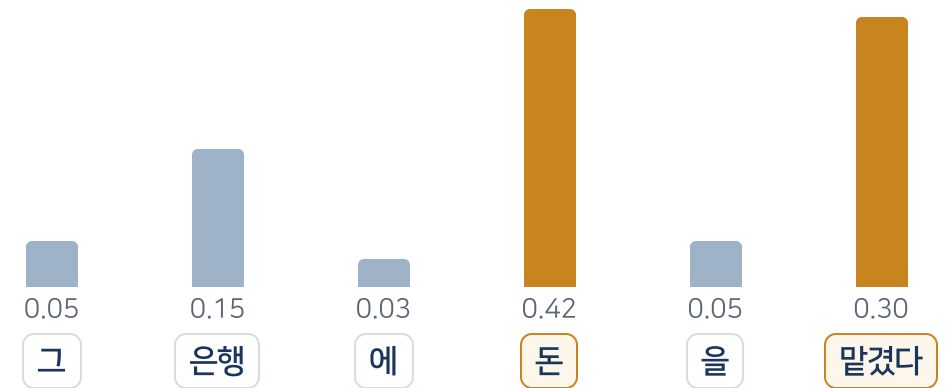
◆ 작동 방식 — 세 단계

- ① Attention Score: 각 단어를 Query·Key·Value 세 벡터로 변환하고, Query와 모든 Key의 유사도(내적)를 계산
- ② 정규화: Softmax로 유사도를 합이 1인 가중치로 바꾼다 — '어디에 얼마나 주목할지'
- ③ 가중합: 가중치대로 Value를 섞어 새 표현을 만든다 — 문맥이 반영된 단어 벡터

◆ Multi-Head: 이 계산을 여러 관점(head)에서 병렬로 수행해 결합

- 한 head는 문법 관계, 다른 head는 지시 관계... 서로 다른 종류의 관계를 동시에 본다

Query: '은행' → 어느 단어에 주목(attend)할까?



Key와의 유사도 → softmax → 가중치 (합 = 1)

'돈'과 '없었습니다'에 크게 주목 → '은행'은 강둑(river bank)이 아닌 금융기관으로 재표현된다. 출력 = $\sum \text{가중치} \times \text{Value}$.
모든 단어가 동시에 이 계산을 한다 (자체 도식)

2. Transformer의 혁신 — 그리고 세 갈래의 파생

◆ 병렬 처리 가능

- Self-Attention 계산이 모든 위치에서 동시에 수행 — GPU·TPU 같은 병렬 하드웨어에서 효율적

◆ 모델 확장성

- 레이어 수와 Attention head 수를 늘리면 그대로 커진다 — '규모의 시대'를 여는 조건

◆ 같은 블록, 세 가지 사용법

- 인코더만 (BERT): 이해 중심 — 네 태스크
- 디코더만 (GPT): 생성 중심 → 오늘의 LLM — 여섯 태스크
- 둘 다 (T5): 텍스트→텍스트 — 여섯 태스크, 이번 실습의 세 번째 방식

Encoder-only	Decoder-only	Encoder-Decoder
BERT (2018) 문장을 양방향으로 읽어 '이해'에 강함 → 라벨·스팬을 직접 예측	GPT (2018~) · 오늘의 LLM 왼쪽→오른쪽 생성 지시문을 받아 답을 '생성' → 모든 태스크를 생성으로	Transformer 원형 · T5 (2019) 입력을 읽고 출력을 생성 번역·요약에 자연스러움 → '텍스트→텍스트' 통일
실습: 네 태스크	실습: 여섯 태스크	실습: 여섯 태스크

Transformer 블록을 어떻게 쓰는가에 따른 세 갈래
(자체 도식)

2. 같은 태스크를 세 방식이 어떻게 푸는가

태스크	BERT 인코더	T5 인코더-디코더	GPT 계열 디코더
주제분류	[CLS] 벡터로 7개 분야 중 하나를 고른다	'정치' 라는 글자를 쓴다	지시문을 읽고 '정치' 라고 쓴다
문장유사도	[CLS] 벡터로 0~5 점수 하나를 낸다	'3.8' 이라는 글자를 쓴다	지시문을 읽고 '3.8' 이라고 쓴다
개체명인식	토큰마다 태그를 붙인다 (B-PS/I-PS...)	개체 목록을 텍스트로 쓴다	JSON 배열로 쓴다
기계독해	정답의 시작·끝 위치를 고른다	정답 문자열을 쓴다	정답 문자열을 쓴다
학습 단위	태스크마다 head와 모델을 따로	태스크마다 모델을 따로 (코드는 하나)	모델 하나 + 어댑터 하나

출력이 '고르기'에서 '쓰기'로 옮겨 간다. 이 차이가 뒤에 나오는 구현 비교표와 형식 오류율의 뿌리다.

2. 고르기로는 풀 수 없는 문제 — 수학추론과 SQL생성

◆ 수학추론 (GSM8K-ko) — 정답이 계산의 결과다

- 문제: "연필 세 다스를 사서 다섯 자루를 잃어버렸다. 남은 연필은 몇 자루인가?" (예시)
- 정답: 풀이 몇 줄을 쓰고 마지막 줄에 ##### 31
- 31이라는 숫자는 문제 어디에도 없다 — 고를 라벨도, 가리킬 구간도 없다

◆ SQL생성 (Spider-ko) — 정답이 새로 짜는 문장이다

- 입력: 데이터베이스 스키마 + "인구가 가장 많은 도시 세 곳은?" (예시)
- 정답: `SELECT name FROM city ORDER BY population DESC LIMIT 3`
- 질문에 없는 열 이름·정렬·개수 제한을 조합해서 만들어야 한다

◆ 그래서 인코더로는 시도조차 할 수 없다

- 분류 head는 미리 정한 라벨 중에서만 고르고, 스펠 head는 입력 안의 구간만 가리킨다
- 이 두 태스크는 T5(인코더-디코더)와 GPT 계열(디코더)만 후보가 된다 — 수학추론은 시간이 남을 때 다루는 보너스다

3. BERT의 혁신

◆ BERT (Bidirectional Encoder Representations from Transformers, Google 2018)

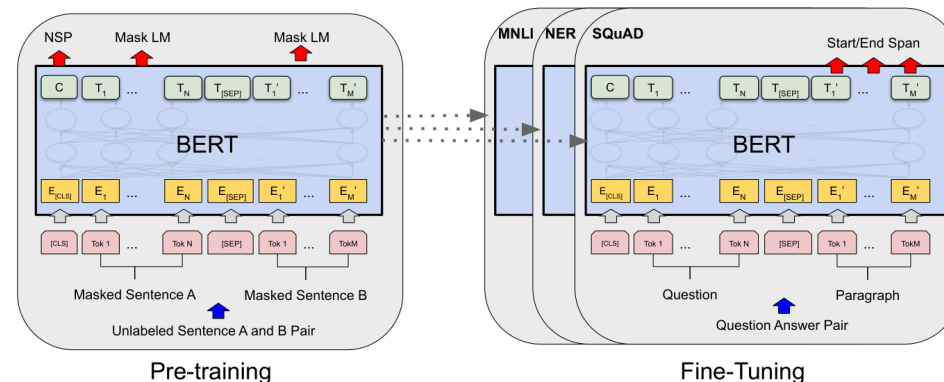
- Transformer 인코더만 사용, 문장을 양방향으로 읽어 문맥을 더 깊이 이해하는 모델
- 11개 언어이해 벤치마크에서 당시 최고 성능 — '사전학습 + 미세조정'을 표준으로 만들었다

◆ 기본 구조와 입력 표현

- 단어 임베딩(Token) + 위치 임베딩(Position) + 세그먼트 임베딩(Segment: 문장 A/B 구분)의 합
- [CLS] 토큰: 문장 전체를 대표하는 자리 — 분류 문제의 출력으로 사용
- [SEP] 토큰: 두 문장(질문/지문, 유사도를 재는 문장 쌍)을 구분

◆ 규모

- BERT-Base 1.1억(110M), BERT-Large 3.4억(340M) 파라미터 — 오늘 실습의 [blue/roberta-base](#)가 이 크기



BERT 사전학습과 미세조정 (Figure 1) · 출처: [2] Devlin et al., 2019

3. BERT의 사전학습 — 정답표 없이 언어를 배우는 법

- ◆ 사전학습 (Pre-training): 대용량 텍스트에서 자기지도(Self-Supervised) 방식으로 학습
 - 사람이 라벨을 달지 않는다 — 텍스트 자체에서 문제와 정답을 자동으로 만든다
- ◆ Masked Language Model (MLM)
 - 입력 문장의 일부(15%) 단어를 [MASK]로 가리고 그 단어를 맞히게 한다
 - 예: "나는 [MASK]에 갔다" → '학교' — 양쪽 문맥을 모두 보므로 GPT식 한 방향 예측보다 이해에 유리
- ◆ Next Sentence Prediction (NSP)
 - 두 문장이 이어지는 문장인지 예측 — 문장 간 관계 학습 (예: "오늘 날씨가 좋다." / "산책을 가자." → 연관 있음)
 - 후속 연구(RoBERTa 등)에서는 효과가 작다고 보고 제거 — 실습 모델 `klue/roberta-base`도 NSP 없이 학습
- ◆ 데이터: 위키백과 + 책 코퍼스 약 33억 단어 — 당시로는 대규모, 지금 LLM(수조 토큰)의 1/1000 이하

3. BERT의 미세조정 — 오늘 오후에 직접 하는 것

◆ 미세조정 (Fine-tuning)의 효과

- 사전학습된 BERT 위에 태스크별 출력층(head)을 얹고, 해당 태스크 데이터로 전체 모델을 추가 학습
- 같은 BERT를 주제분류·문장유사도·개체명인식·기계독해에 모두 적용 가능하고 높은 성능 유지 — 전이학습(Transfer Learning) 효과

◆ 태스크별로 달라지는 것 (1일차 실습)

- 주제분류: [CLS] 벡터 → 선형층 → 클래스 확률 (KLUE-YNAT, 뉴스 제목의 7개 분야)
- 문장유사도: [CLS] 벡터 → 선형층 → 0~5점 (KLUE-STs, 회귀)
- 개체명인식: 토큰마다 → 선형층 → 개체 태그 (KLUE-NER, B-PS/I-PS...)
- 기계독해: 토큰마다 → 정답 시작/끝 위치 점수 (KorQuAD)

◆ BERT 방식의 특징 (T5·GPT 계열과 비교하기 위해 기억할 것)

- 태스크마다 모델 구조(head)가 다르고, 태스크마다 별도의 모델 파일이 생긴다
- 전체 파라미터(1.1억)를 모두 갱신 — 학습 데이터가 수만 건이면 GPU 한 장으로 수 분~수십 분
- 출력은 '확률/위치'이며 사람이 읽을 문장이 아니다 — 형식이 깨질 일이 없다는 장점

4. T5의 통합적 접근

◆ T5 (Text-to-Text Transfer Transformer, Google 2019)

- 모든 자연어처리 태스크를 "텍스트 입력 → 텍스트 출력"으로 통일한 접근

◆ 기존 방식의 한계와 T5의 해결

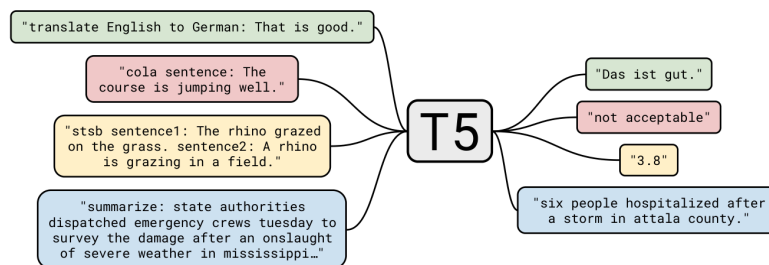
- 기존: 분류는 클래스 라벨, 생성은 텍스트, 번역은 다른 언어 — 태스크마다 다른 출력 형식과 구조
- T5: 앞에 태스크 접두어를 붙인 텍스트를 넣고 텍스트로 답한다
- 예) "translate English to German: That is good." → "Das ist gut." / "cola sentence: ..." → "acceptable"

◆ 통합의 힘

- 하나의 모델로 다양한 태스크 수행, 태스크 간 지식 전이, 새 태스크 추가가 매우 간단

◆ 이것이 실습의 발상이다 — T5는 배경이자 실제 기준선이다

- 우리도 여섯 태스크(주제분류·문장유사도·개체명인식·기계독해·수학추론·SQL생성)를 전부 '지시문 + 입력 → 답 텍스트'로 바꿔 한 모델에 넣는다
- T5 계열 한국어 모델 [paust/pko-t5-base](#)는 실습에서 인코더-디코더 기준선으로 직접 학습해 BERT·GPT 계열과 나란히 비교한다



T5의 텍스트→텍스트 프레임워크 (Figure 1) · 출처: [3] Raffel et al., 2020

4. T5의 기술적 특징과 역사적 의의

◆ 기술적 특징

- Transformer Encoder-Decoder 구조 — BERT의 양방향 인코더와 GPT식 자기회귀 디코더의 결합
- Span Corruption 사전학습: 연속된 토큰 구간을 가리고 복원 — MLM의 확장

◆ 학습 데이터와 규모 실험

- C4 (Colossal Clean Crawled Corpus): 웹에서 수집·정제한 약 750GB 텍스트 — 데이터 정제의 중요성을 체계적으로 보임
- 멀티태스크 혼합 학습: 번역·요약·질의응답·분류 등 여러 태스크를 비율을 조정해 동시에 학습
- Small(60M) · Base(220M) · Large(770M) · XL(3B) · XXL(11B) — 크기를 키울수록 일관되게 향상

◆ 역사적 의의

- "명령 텍스트 입력 → 텍스트 출력"이 이후 ChatGPT식 지시 따르기(Instruction Following)의 개념적 기반
- 하나의 모델로 여러 업무를 처리하는 '범용 모델' 설계 철학의 선구자 — 이후 FLAN-T5로 확장

5. GPT 시리즈의 등장 — GPT-1

◆ GPT (Generative Pre-trained Transformer, OpenAI 2018)

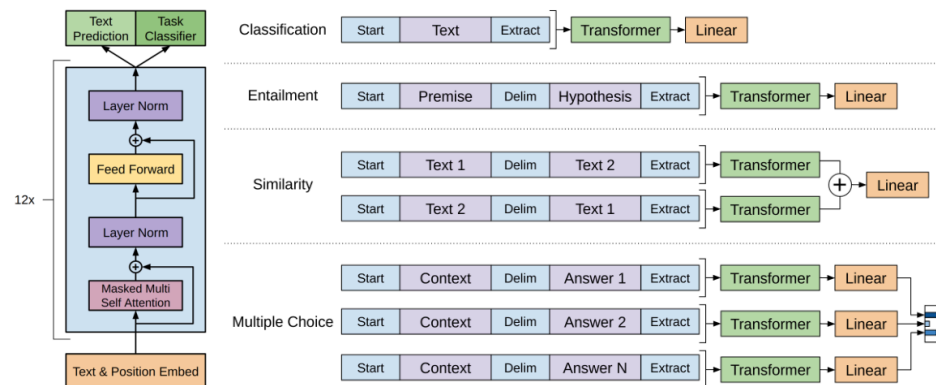
- 생성적 사전학습(Generative Pre-training) + 태스크별 미세조정을 제안한 언어모델

◆ 기본 구조

- Transformer 디코더만 사용 (12층, 1.17억 파라미터) — BERT-Base와 비슷한 크기
- 단방향(왼→오) 모델: 앞의 단어들로 다음 단어를 예측하며 학습 — 자연스럽게 '생성'이 된다
- BPE (Byte Pair Encoding) 토큰라이저: 자주 나오는 글자 조합을 합쳐 서브워드 단위로 분할

◆ 미세조정 방식 (그림 오른쪽)

- 분류·함의·유사도·선택형 문제를 모두 '텍스트를 이어 붙여 디코더에 넣는' 형태로 변환
- BERT와 같은 해에 나왔고 당시에는 BERT에 성능이 밀렸다 — 그러나 '생성'이라는 방향은 이후 LLM의 본류가 된다



GPT-1 구조와 태스크별 입력 변환 (Figure 1) · 출처: [4] Radford et al., 2018

5. GPT-2 — 규모를 키우면 무슨 일이 생기는가

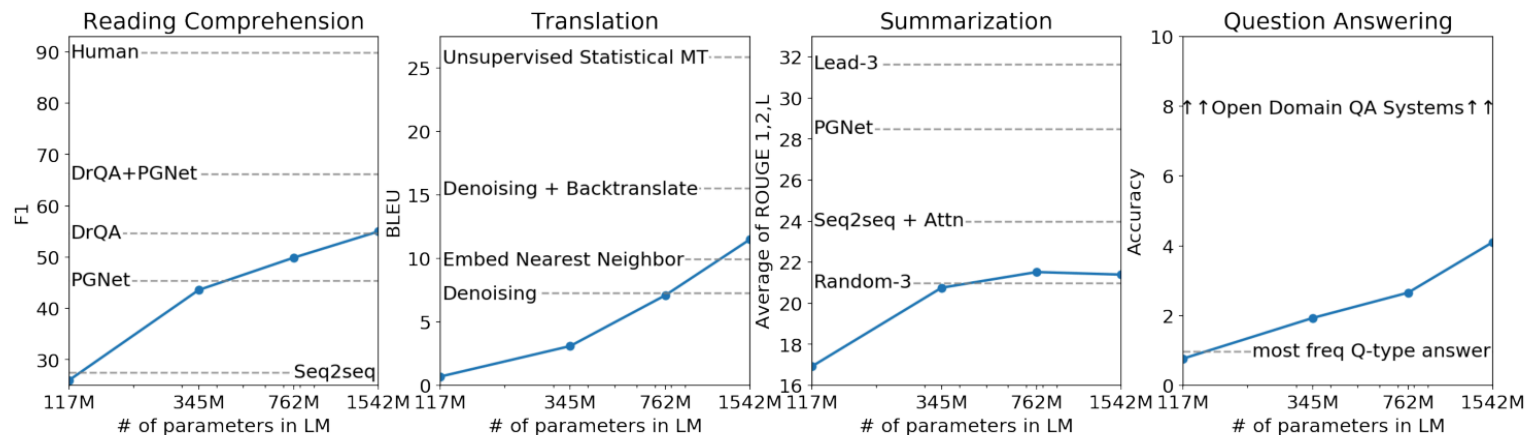
◆ GPT-2 (OpenAI, 2019)

- GPT-1의 구조를 확장하여 최대 15억(1.5B) 파라미터, 48층 — 10배 이상 커졌다
- WebText: Reddit에서 추천을 받은 링크의 웹 문서 약 800만 건(40GB) — 데이터 품질 필터링

◆ 주요 혁신

- 규모의 효과 입증: 모델이 커질수록 언어모델 성능(perplexity)과 다운스트림 성능이 함께 향상 (그림)
- 제로샷(Zero-Shot): 별도 미세조정 없이 프롬프트만으로 독해·번역·요약·질의응답을 시도
- "Fine-tuning 없이도 작동할 수 있다"는 가능성을 처음 보였으나, 복잡한 문제에서는 여전히 약했다

◆ 당시의 논란: 너무 그럴듯한 글을 써서 위험하다며 가장 큰 모델의 공개를 늦춤 — 'AI 안전' 논의의 시작점



모델 크기에 따른 제로샷 성능 (Figure 1) · 출처: [5] Radford et al., 2019

5. GPT-3 — 문맥 내 학습(In-Context Learning)의 발견

◆ GPT-3 (OpenAI, 2020)

- 1,750억(175B) 파라미터 — GPT-2의 100배, 당시 최대 규모. 학습 데이터 약 3,000억 토큰

◆ 핵심 혁신: 문맥 내 학습 (In-Context Learning)

- 가중치 업데이트 없이, 프롬프트에 적은 지시와 예시만으로 새로운 작업을 수행하는 패러다임

◆ 프롬프트 제공 방식에 따른 성능

- Zero-shot: 예시 없이 지시만 — GPT-2 대비 월등한 향상
- One-shot / Few-shot: 예시 1개 또는 몇 개를 함께 제시
- 예시를 줄수록 성능이 비약적으로 상승하여 미세조정 모델에 맞먹는 수준에 도달

◆ 의미: '학습'의 뜻이 바뀌었다

- 가중치를 바꾸는 학습(fine-tuning) 없이도 프롬프트만으로 모델이 '배운 것처럼' 행동
- 프롬프트 엔지니어링이라는 새 분야의 출발점

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.



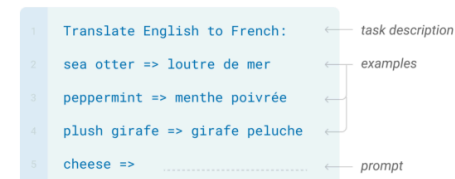
One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.



Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



[참고] 규모의 법칙과 창발 능력 — '더 크게'가 답이 된 이유

◆ 스케일링 법칙 (Scaling Laws, Kaplan et al. 2020)

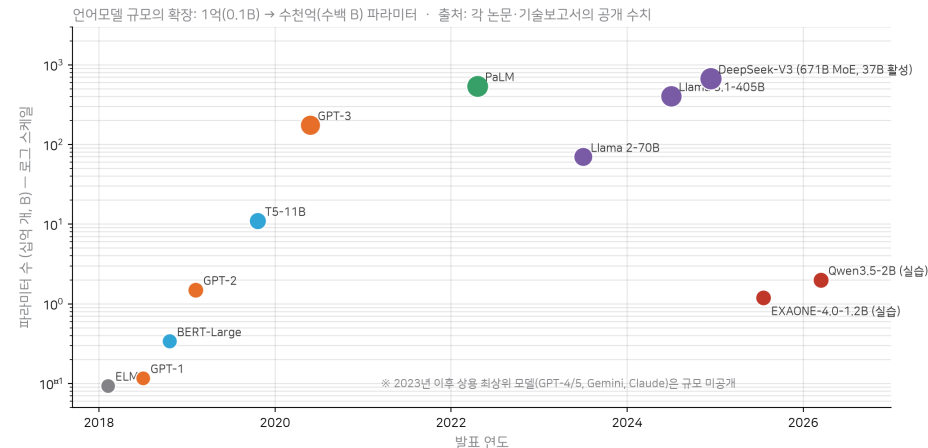
- 파라미터 수·데이터 양·연산량을 키우면 손실(loss)이 거듭제곱 법칙(power law)을 따라 예측 가능하게 감소
- "얼마나 키우면 얼마나 좋아지는가"를 미리 계산할 수 있게 됨 → 대규모 투자의 근거

◆ 창발 능력 (Emergent Abilities, Wei et al. 2022)

- 작은 모델에서는 거의 0점이던 능력(다단계 산술, 단어 퍼즐 등)이 특정 규모를 넘으면 갑자기 나타남
- '연속적 개선'이 아닌 '상전이'처럼 보이는 현상 — 측정 지표에 따른 착시라는 반론도 있음 (Schaeffer et al. 2023)

◆ 그 다음 질문

- 규모 경쟁이 한계에 부딪히자, '학습 규모' 외에 '추론 시 연산(Test-time Compute)'과 '정렬(Alignment)'이 새 축이 된다 → Part 2 후반



모델 규모의 확장 (자체 작성, 각 논문·기술보고서 공개 수치)

[참고] LLM 작동 원리를 시각적으로 이해하기

◆ 추천 영상: "How Large Language Models Work" (3Blue1Brown)

- youtube.com/watch?v=HnvitMTkXro
- LLM(ChatGPT)의 학습 및 추론 과정을 애니메이션으로 설명 — 오늘 강의 복습용으로 권장

◆ 함께 보면 좋은 자료

- Jay Alammar, "The Illustrated Transformer" (jalammar.github.io) — Self-Attention 그림 설명의 고전
- Hugging Face LLM Course (huggingface.co/learn) — 실습 라이브러리(Transformers·TRL·PEFT) 공식 입문
- Do it! BERT와 GPT로 배우는 자연어처리 (ratsgo.github.io/nlpbook) — 1일차 실습 구성의 참고서

◆ 내일 실습 전에 10분만 투자하면 좋은 것

- 위 3Blue1Brown 영상 1편 시청 — '토큰', '다음 단어 예측', '어텐션'이 그림으로 정리된다

PART
III

평가와 최신 모델, 그리고 핵심 요소기술

- 1 언어모델의 평가 — 무엇을, 어떻게 재는가
- 2 상용 LLM의 현황: OpenAI · Google · Anthropic
- 3 오픈소스(오픈웨이트) LLM과 한국어 LLM 생태계
- 4 ChatGPT를 만든 핵심 요소기술 — RAG · 지시 미세조정 · RLHF · CoT · 추론 · 초장문 · LoRA · MoE



6. 언어모델의 평가 — 다섯 개의 축

평가 축	묻는 것	대표 벤치마크	현재 상태 (2026)
지식·이해	폭넓은 교과 지식을 아는가	MMLU [7] (57과목 4지선다) → MMLU-Pro [8] (10지선다, 추론형)	MMLU는 최상위 모델이 90%대 포화 → Pro·GPQA로 이동
전문가 수준 추론	박사급 전문가도 어려운 문제	GPQA Diamond [9] (검색으로 못 푸는 대학원 과학 문제)	최상위 모델이 전문가(약 70%)를 넘어 90% 안팎
수학	다단계 수식 추론	MATH [10] (경시대회) → AIME, IMO	AIME 90%대, IMO 금메달 수준(2025, Deep Think·o-계열)
코딩	코드 생성·실제 이슈 해결	HumanEval [11] (함수 완성) → SWE-bench Verified [12] (실제 GitHub 이슈)	HumanEval 포화; SWE-bench Verified 80% 안팎 (2026)
최종 시험	인류 지식의 최전선	Humanity's Last Exam [13] (2,500문항, 100여 과목 전문가 출제)	출제 당시 10% 미만 → 2026 최상위 40%대, 아직 미포화

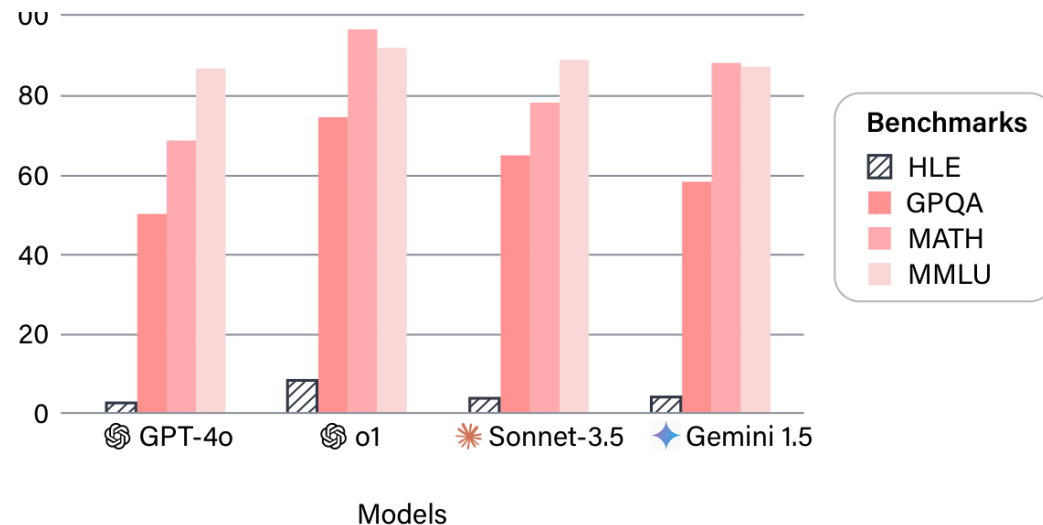
6. 벤치마크 두 가지 — SWE-bench와 Humanity's Last Exam

◆ SWE-bench (2024): 실제 GitHub 이슈를 해결하는가

- 입력은 이슈 설명과 저장소 전체, 출력은 코드 수정(PR). 숨겨진 단위 테스트가 통과하면 성공
- '함수 하나 채우기'(HumanEval)에서 '실제 소프트웨어 개발'로 평가의 무게가 옮겨간 상징
- 2024년 초 최고 모델이 한 자리 수 % → 2026년 Verified 부분집합에서 80% 안팎

◆ Humanity's Last Exam (2025~): 전문가가 낸 최전선 문제

- 2,500문항, 수학·물리·인문 등 100여 과목, 전 세계 전문가 1,000명이 출제·검증
- 그림: 2025년 1월 발표 당시 — 기존 벤치마크(MMLU·MATH·GPQA)는 60~90%인데 HLE는 10% 미만
- 2026년 현재 최상위 모델이 40%대까지 도달 — '벤치마크 포화 → 대체' 주기가 계속된다



HLE 발표 당시 최상위 모델 성능 · 출처: [13] Phan et al., 2025

6. 성능표를 읽을 때 확인할 다섯 가지

- ◆ ① 같은 조건인가? — 도구 사용(검색·코드 실행) 여부, 추론 예산(thinking 시간), 시도 횟수(`pass@1` vs `pass@k`)가 같아야 비교 가능
- ◆ ② 누가 측정했는가? — 개발사 자체 발표(technical report)와 제3자 리더보드(LMArena, Artificial Analysis, Epoch AI)의 숫자는 종종 다르다
- ◆ ③ 포화되었는가? — 90%를 넘은 벤치마크의 1~2점 차이는 잡음일 수 있다. 그 축에서는 더 어려운 후속 벤치마크를 본다
- ◆ ④ 오염되지 않았는가? — 평가 문제가 학습 데이터에 섞였을 가능성(contamination). 새로 만든 비공개 문제(예: 매년 갱신되는 AIME)로 교차 확인
- ◆ ⑤ 우리 일과 관련 있는가? — 한국어·우리 도메인·우리 형식의 자체 평가셋 100건이 공개 벤치마크 점수보다 결정에 더 유용하다
- ◆ 이를 실습에서 이 원칙을 그대로 지킨다
 - 학습셋과 평가셋을 엄격히 분리(공식 validation 사용, 학습에 절대 포함하지 않음) · 학습 전/후, 그리고 BERT·T5·GPT 계열 세 방식을 같은 문항·같은 지표로 측정 · 형식 오류율(`broken_format`)도 함께 보고

7. GPT-3에서 ChatGPT로 — 무엇이 달라졌는가

◆ GPT-3.5 계열 (2022): 코드 학습 + 지시 따르기 + 인간 정렬

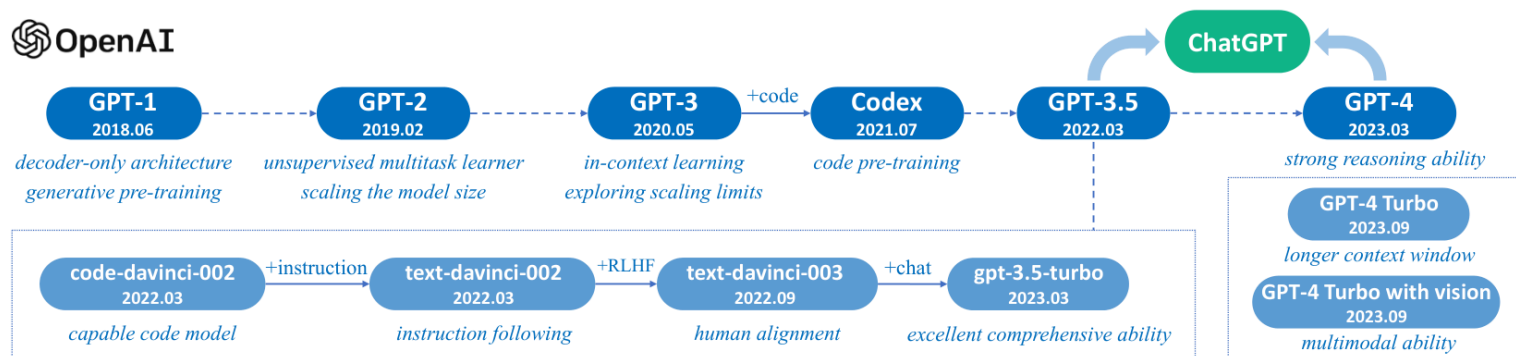
- Codex(2021): 코드로 학습 → 논리·추론 능력이 함께 향상되는 부수 효과
- InstructGPT(2022.03): 인간 피드백 강화학습(RLHF)으로 '지시를 따르고 유용하며 무해한' 답 → 요소기술 절에서 자세히
- ChatGPT(2022.11): 대화 형식으로 미세조정. 출시 5일 만에 사용자 100만, 2개월에 1억 — 역대 최속

◆ 🔑 핵심 통찰: 규모(GPT-3)만으로는 부족했다

- 175B 모델이 있어도 '사람이 원하는 답'을 하게 만드는 정렬(Alignment) 없이는 제품이 되지 않았다
- 다음 단어 예측기 → 대화 상대: 능력이 아니라 '태도'를 학습시킨 것이 전환점

◆ GPT-4 (2023.03): 멀티모달 + 신뢰성

- 이미지 입력, 32K 문맥, 미국 변호사 시험 상위 10% — 모델 규모·학습 세부는 처음으로 비공개



GPT 계열의 기술적 진화 (Figure 4) · 출처: [54] Zhao et al., 2023

7. 추론 모델의 등장 — o1, o3 그리고 '생각하는 시간'

◆ OpenAI o1 (2024.09): 답하기 전에 생각한다

- 강화학습으로 긴 사고 사슬(Chain of Thought)을 스스로 만들도록 학습 — 사용자에게는 '생각 중...'으로 보임
- 수학 올림피아드(AIME) 정답률 GPT-4o 13% → o1 83%, 코딩·과학에서 박사급 성능
- 새 스케일링 축: 학습 연산이 아닌 **추론 시 연산(Test-time Compute)**을 늘리면 성능이 오른다

◆ o3 · o4-mini (2025.04): 추론 + 도구 사용

- 생각하는 도중에 웹 검색·코드 실행·이미지 분석 도구를 스스로 호출 — '에이전트형 추론'
- 이미지를 사고 과정 안에서 다룸(확대·회전) — 시각적 추론

◆ 추론 모델의 대가

- 응답이 느리고 비싸다 → 이후 모든 회사가 '빠른 모드 / 생각 모드'를 한 모델에 합치는 하이브리드로 수렴
- 도입 데모에서 수학 문제에 '생각 중'이 길어진 것이 바로 이것

7. GPT-5 시리즈 (2025.08 ~ 2026) — 라우터, 에이전트, 컴퓨터 사용

모델	출시	핵심 변화
GPT-5	2025.08	라우터가 질문 난이도에 따라 Instant/Thinking 모델을 자동 선택. '안전한 완성(safe completion)' 정책. 무료 사용자에게도 개방
GPT-5.1 / 5.2	2025.11 / 12	대화 톤·성격 조정, 추론 효율 개선. 5.2는 전문 지식·에이전트 작업(문서·스프레드시트) 강화 [17]
GPT-5.3-Codex	2026.02	코딩 에이전트 특화 — 장시간 자율 작업, 저장소 단위 리팩터링 [18]
GPT-5.4	2026.03	컴퓨터 사용(화면을 보고 마우스·키보드 조작) 내장. OSWorld-Verified 75% (사람 평균 72%, 5.2는 47%). 사실 오류 5.2 대비 약 1/3 감소
GPT-5.5 (+Pro)	2026.04	에이전트가 브라우저·문서·표·코드 도구 사이를 스스로 오가며 장시간 업무 수행. 'Pro'는 최대 추론 예산
GPT-5.6	2026.07	용량·비용별 등급(3단계) 체계로 재편, 최상위 등급이 코딩 플래그십. 자세한 성능 수치는 공식 발표 기준으로 확인 필요

7. GPT Image — 이미지 생성이 언어모델 안으로

◆ GPT-4o 이미지 생성 (2025.03) → GPT Image 시리즈

- 별도의 그림 모델(DALL·E)이 아니라 언어모델 자체가 이미지를 토큰처럼 생성 — '네이티브 멀티모달 출력'
- 텍스트 렌더링(글자가 들어간 포스터·도표), 대화로 이어가는 수정("배경만 바꿔줘"), 여러 이미지 조합

◆ 의미

- 이해(입력)뿐 아니라 생성(출력)도 텍스트·이미지가 한 모델에서 나온다
- Google의 Nano Banana(Gemini 이미지, 2025.08~), Nano Banana Pro/2(2025.11, 2026.02)와 함께 '대화형 이미지 편집'이 표준이 됨

◆ 교육·업무 활용 예

- 강의 슬라이드 삽화, 데이터 도표 초안, 문서 속 그림 수정 — 단, 저작권·초상권 확인은 사용자 책임

7. Google Gemini — 네이티브 멀티모달과 초장문

모델	출시	핵심 변화
Gemini 1.0 / 1.5	2023.12 / 2024.02	처음부터 텍스트·이미지·오디오·영상을 함께 학습한 네이티브 멀티모달. 1.5 Pro는 100만~200만 토큰 문맥 (책 수십 권)
Gemini 2.0 / 2.5	2024.12 / 2025.03	에이전트 지향(도구 사용), 2.5 Pro는 '생각하는 모델'로 전환. Deep Think 모드가 2025 IMO 금메달 기준 달성
Gemini 3 Pro / Deep Think / 3 Flash	2025.11 ~ 12	희소 MoE, 100만 토큰 입력. 발표 당시 HLE 37.5%(도구 없이)·41%(검색+코드), GPT-5 Pro 31.6% 대비 우위 [20]. Flash는 속도·비용 최적화
Gemini 3.1 Pro	2026.02	추론·코딩 개선, Nano Banana 2 이미지 모델 동반
Gemini 3.5 / 3.6 / 3.7 Flash	2026.05 / 07 / 08	경량 계열을 매달 갱신 — 대량 처리·저비용 수요. 3.5 Pro는 2026.09 현재 미출시(파트너 테스트 보도)

7. Anthropic Claude — 안전 연구에서 시작해 코딩 에이전트로

모델	출시	핵심 변화
Claude 3 / 3.5 Sonnet	2024.03 / 06	헌법적 AI(Constitutional AI)로 정렬. 3.5 Sonnet부터 코딩 성능으로 주목, 컴퓨터 사용 베타(2024.10)
Claude 3.7 Sonnet	2025.02	첫 하이브리드 추론(extended thinking) — 빠른 답과 긴 생각을 한 모델에서 선택
Claude 4 (Opus/Sonnet)	2025.05	Claude Code(터미널 코딩 에이전트) 정식 출시. 장시간 자율 작업, SWE-bench 최상위권
Sonnet 4.5 → Opus 4.5	2025.09 → 11	메모리 도구·컨텍스트 편집(장기 작업), 노력(effort) 조절 파라미터
Opus 4.6 / 4.7 / 4.8	2026.02 / 04 / 05	128K 출력, 적응형 추론, 에이전트 팀(여러 에이전트 협업) — 두 달 간격 갱신
Fable 5 · Mythos 5 · Sonnet 5 · Opus 5	2026.06 ~ 07	Opus 위의 새 등급(Fable/Mythos). Mythos 5는 사이버 방어 협력 프로그램(Project Glasswing)에 한정 배포. Opus 5(07.24)는 Fable 5 근접 성능을 절반 가격으로 [41]

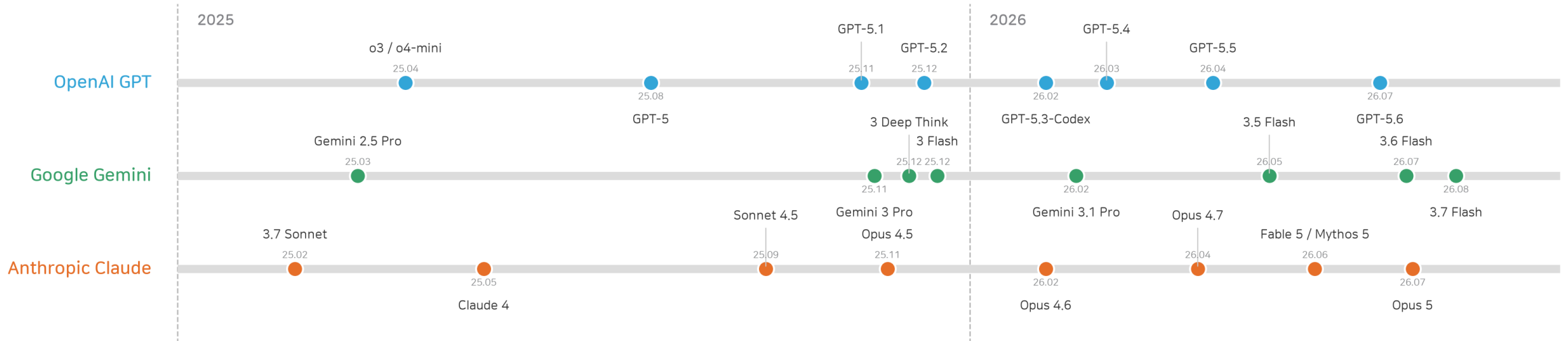
7. 상용 3사 비교 — 2026년 9월 시점의 큰 그림

항목	OpenAI (GPT)	Google (Gemini)	Anthropic (Claude)
강점	가장 넓은 사용자층, 라우터로 쉬운 사용, 이미지·음성·컴퓨터 사용 통합	네이티브 멀티모달(영상·음성), 100만 토큰 초장문, 검색·Workspace 통합, 저비용 Flash	코딩·에이전트 작업의 신뢰성, 긴 자율 작업, 안전·정렬 연구
추론 방식	라우터가 Instant/Thinking 자동 배정, Pro는 최대 예산	Thinking 기본 + Deep Think(최상위, 병렬 사고)	하이브리드(빠른 답/확장 사고), 적응형 추론, effort 조절
도구·에이전트	Codex(코딩), Operator/컴퓨터 사용, 에이전트 모드(브라우저·문서·표)	Gemini CLI·Antigravity(개발), Agent Mode, Workspace 에이전트	Claude Code(터미널), 컴퓨터 사용, 에이전트 팀, MCP(도구 연결 표준) 제안
추천 용도	범용 업무 보조, 이미지 생성, 일반 사용자 대상 서비스	긴 문서·영상 분석, 대량 저비용 처리, Google 생태계	코드 작성·리뷰·리팩터링, 장문 작성, 규정·안전이 중요한 업무
오픈웨이트	gpt-oss (2025.08, 120B/20B)	Gemma 4 (2026, Apache 2.0, E2B~31B)	없음 (폐쇄 모델만)

7. 상용 LLM의 공통 트렌드 (2025 ~ 2026)

- ◆ ① 추론(Reasoning)이 기본 사양 — 모든 플래그십이 '생각하는 모드'를 갖고, 하이브리드(빠른 답/긴 생각)로 수렴
- ◆ ② 에이전트(Agent) — 답을 '말하는' 모델에서 도구를 쓰며 '일하는' 모델로: 코딩 → 브라우저 → 컴퓨터 조작 → 여러 에이전트 협업
- ◆ ③ 네이티브 멀티모달 — 텍스트·이미지·음성·영상을 입력도 하고 출력도 한다 (GPT Image, Nano Banana)
- ◆ ④ 초장문 + 메모리 — 100만 토큰 문맥, 대화 간 기억, 컨텍스트 편집 — '한 번의 질문'에서 '함께 일하는 세션'으로
- ◆ ⑤ 갱신 주기 단축과 등급 세분화 — 1~2개월 간격 출시, 한 세대 안에 Flash/Mini·표준·Pro/Deep Think 등급
- ◆ ⑥ 비공개화와 평가의 어려움 — 규모·데이터 비공개, 벤치마크 포화 → 제3자 평가와 자체 평가셋의 중요성 증가

상용 LLM 주요 모델 출시 흐름 (2025.01 ~ 2026.09) · 출처: 각사 공식 발표 · Wikipedia (2026.09.02 확인)



상용 3사 주요 모델 출시 타임라인 2025.01
~ 2026.09 (자체 작성, 각 사 공식 발표 기준)

8. 오픈소스(오픈웨이트) LLM — LLaMA 이후

◆ LLaMA (Meta, 2023.02): 연구 목적으로 가중치 공개 → 오픈 LLM 생태계의 시작

- 7B~65B, '작은 모델을 더 많은 데이터로' 학습 — 13B 모델이 GPT-3(175B)를 여러 벤치마크에서 앞섬
- Llama 2(2023.07) 상업적 사용 허용, Llama 3(2024) 8B/70B/405B, Llama 4(2025.04) MoE·초장문
- Alpaca·Vicuna 등 파생: 수만 건의 지시 데이터로 미세조정하면 ChatGPT에 근접 — 지시 미세조정의 대중화

◆ '오픈소스'와 '오픈웨이트'

- 가중치는 공개하지만 학습 데이터·코드는 비공개인 경우가 많다 → 정확히는 '오픈웨이트'
- 라이선스 확인 필수: Apache 2.0·MIT(자유) / Llama·Gemma 자체 라이선스(조건부) / 연구 전용

◆ 왜 오픈웨이트가 중요한가

- 내 서버에서 실행(데이터 외부 유출 없음), 내 데이터로 미세조정, 비용 통제, 연구 재현 — 내일 실습이 가능한 이유

8. 주요 오픈웨이트 모델 (2025 ~ 2026)

계열 (개발사)	최신	특징	라이선스
Llama (Meta)	Llama 4 (2025.04)	MoE 구조, Scout는 1,000만 토큰 문맥. 오픈 생태계의 출발점	Llama 라이선스 (조건부)
Gemma (Google)	Gemma 4 (2026)	E2B·E4B(온디바이스)·12B·26B-A4B(MoE)·31B. Gemini 기술 이전. E2B는 내일 비교 대상	Apache 2.0
Qwen (Alibaba)	Qwen3.5 ~ 3.8 (2026)	0.8B부터 대형까지 폭넓은 크기, 다국어(한국어 포함) 강함, 하이브리드 추론. 2B는 내일 비교 대상	Apache 2.0
DeepSeek (중국)	DeepSeek-V4 (2026)	대규모 MoE를 저비용으로 학습·공개(R1, 2025.01이 계기). 코딩·수학 강점	MIT
gpt-oss (OpenAI)	120B / 20B (2025.08)	OpenAI 첫 오픈웨이트(GPT-2 이후). 추론 모델, 단일 GPU(20B) 실행	Apache 2.0
Mistral (프랑스)	Medium 3.5 등 (2026)	유럽 대표. 효율적 중형 모델, 코드(Codestral)	Apache 2.0 / 자체
Phi (Microsoft)	Phi-4 계열 (2025)	소형 고품질 — '교과서 품질 데이터'로 작은 모델의 추론 향상	MIT

8. 한국어 LLM 생태계 (2025 ~ 2026)

◆ 독자 AI 파운데이션 모델 프로젝트 (과기정통부·NIPA, 2025.08 ~)

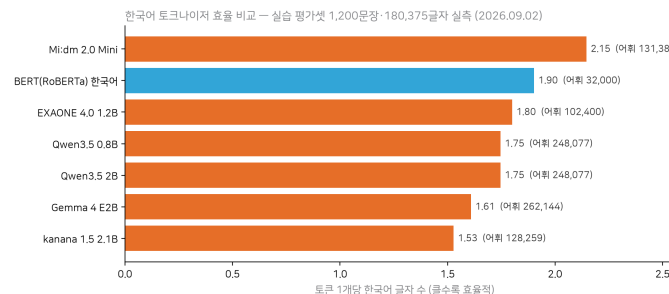
- 국가 주도로 5개 컨소시엄 선정 → 단계 평가로 압축. 2차 평가(2026.08.18): 업스테이지(Solar Open2)·SKT(A.X K2)·LG AI연구원(K-EXAONE 2.0) 진출
- 평가 기준: 벤치마크 40 · 전문가 35 · 사용자 25. GPU(B200) 지원 확대

◆ 소형 공개 모델 — 내일 실습에서 실제로 비교한 것

- EXAONE-4.0-1.2B (LG, 2025.07) · kanana-1.5-2.1B (카카오, Apache 2.0) · Mi:dm 2.0 Mini 2.3B (KT, MIT) · HyperCLOVA X SEED 1.5B (네이버)
- A.X 4.0 Light 7B (SKT), 그리고 해외 다국어 모델 Qwen3.5-2B · Gemma 4 E2B

◆ 한국어 처리의 실질적 차이: 토큰나이저 효율 (오른쪽, 실측)

- 같은 1,200문장(18만 글자)을 토큰으로 나뉘었을 때 글자/토큰 — 높을수록 한 토큰에 더 많은 글자
- Mi:dm 2.15 > **klue/roberta** 1.90 > EXAONE 1.80 > Qwen3.5 1.75 > Gemma 4 1.61 > kanana 1.53
- 토큰 수가 많으면 같은 문맥 길이에 덜 담기고 느리고 비싸다 — 한국어 특화 어휘의 가치



한국어 토큰나이저 효율 실측 (KLUE-STS 등 1,200문장 · 자체 측정)

8. 독자 AI 파운데이션 모델 — 국가가 정의한 '독자 모델'

◆ 사업 개요 — 2025년 8월 5개 정예팀 선정

- 서면·발표평가를 거쳐 다섯 팀: 네이버클라우드 · 업스테이지 · SK텔레콤 · NC AI · LG경영개발원 AI연구원 (가나다 순)
- 정부 지원: 데이터 공동구매 100억 원 · 팀별 28억 원 · 방송영상 학습데이터 200억 원, GPU 임차(B200·H100 규모)
- ETRI는 NC AI 정예팀의 참여기관으로 이 사업에 참여했습니다 — 오늘 이 자리에서 드릴 수 있는 이야기입니다

◆ 1차 단계평가 (2026.01)

- 평가 기준은 벤치마크 40점 + 전문가 35점 + 사용자 25점
- LG AI연구원 · SK텔레콤 · 업스테이지 세 팀이 2차 단계로 진출했고, 우리가 참여한 컨소시엄은 그 안에 들지 못했습니다
- 다섯 팀의 모델은 모두 Epoch AI 의 'Notable AI Models' 에 등재됐고, 과기정통부는 1개 정예팀 추가 공모를 추진한다고 밝혔습니다

◆ 이 수업과 이어지는 대목 — 공모안내서의 '독자 모델' 정의

- "해외 모델 미세조정(파인튜닝) 등으로 개발한 파생형 모델이 아니라, 모델의 설계부터 사전학습 과정 등을 수행한 국산 모델"
- 즉 파인튜닝만으로는 파운데이션 모델이 되지 않는다 — 이것이 국가가 세운 기준입니다
- 오늘·내일 우리가 하는 일은 파인튜닝이고, 사전학습은 그 앞 단계입니다. 이 강의의 뼈대(사전학습 → 미세조정)가 여기서 그대로 쓰입니다

8. ETRI 공개 모델로 보는 파인튜닝의 정체

◆ huggingface.co/etri-lirs — 모델 16종을 공개하고 있습니다

- **EAGLE 계열** (3B · 5.4B · 6.7B) 한국어 대화형 · **KEByT5 계열** 토큰 프리 인코더-디코더 — 글자를 토큰으로 나누지 않고 **바이트를 직접** 다룹니다
- KEByT5-base 를 주제분류에 같은 조건(800건·3에폭)으로 학습하면 **59.7%**, 같은 조건 pko-T5-base 는 **80.7%** — 모델이 나빠서라기보다 **바이트 단위 구조가 소규모 학습에서 불리한 특성**으로 읽는 편이 정확합니다

◆ EAGLE-3B 를 내일 실습과 똑같이 학습해 보면 (실측)

- 같은 지시문 데이터 4,800건 · 1 epoch · A6000 **11.4분** · GPU 최대 8.03GB
- **학습 전 여섯 태스크 평균 0.8점**. 지시문을 쥐도 형식을 못 지킵니다 — 주제분류 300건 중 **297건**이 '일곱 라벨 중 하나'라는 형식을 깨뜨렸습니다
- 학습 뒤 형식깨짐은 **297 → 2건**, 평균은 **0.8 → 36.9** — 11분 만입니다
- 절대 점수는 아래쪽입니다. 그런데 이 장이 말하는 것은 끝점이 아니라 **출발점**입니다 — **학습 전 평균 0.8 은 라인업에서 가장 낮습니다** (`kanana-1.5-2.1B-base` 18.9 · `skt/A.X-4.0-Light` 52.0). 그 자리에서 11분 만에 36.9 까지 올라왔습니다

◆ 이 두 사실을 나란히 놓으면 오늘의 요지가 나옵니다

- 파인튜닝은 **형식과 '이 일을 하는 법'을 아주 빠르게** 가르칩니다 — 11분 만에 297건의 형식 오류가 2건이 됩니다
- 그러나 **원래 없던 능력을 만들어 주지는 않습니다** — 수학추론이 2.7에 머무는 것이 같은 이야기입니다
- 9·10번에서 본 '수학추론이 보너스인 이유'와 정확히 이어집니다. **성능표는 한 칸이 아니라 전후와 조건으로 읽습니다**

8. 오픈소스 LLM이 바꾼 생태계

◆ 연구·개발의 민주화

- 대학·중소기업·개인이 최상위권 모델을 내려받아 실험 — 논문 재현과 후속 연구가 가능해짐
- Hugging Face Hub: 모델·데이터셋·평가를 공유하는 사실상의 표준 저장소 (내일 실습도 여기서 내려받는다)

◆ 응용의 다양화

- 의료·법률·금융 등 도메인 특화 미세조정, 온디바이스(스마트폰·노트북) 모델, 기업 내부망 배포

◆ 경쟁 구도의 변화

- 폐쇄 최상위 모델과 오픈 모델의 격차가 6~12개월로 좁혀짐 — 특정 용도에서는 작은 오픈 모델로 충분
- 중국(Qwen·DeepSeek), 유럽(Mistral), 한국(EXAONE·HyperCLOVA X·kanana·Mi:dm·A.X)의 독자 노선

◆ 우리에게 열린 선택지

- API(편리·최신) vs 오픈웨이트 자체 운영(데이터 통제·비용 예측·맞춤 학습) — Part 3에서 선택 기준

9. ChatGPT를 만든 핵심 요소기술 — 개관

요소기술	해결한 문제	대표 연구	2일차 실습과의 관계
RAG (검색 증강 생성)	모델이 모르는·최신 지식, 환각	Lewis et al. 2020 [23]	개념 소개 (실습 범위 밖)
Instruction Tuning (지시 미세조정)	지시를 이해하고 따르기, 새 태스크 일반화	FLAN 2022 [24]	★ 실습의 핵심 — 6태스크 지시문 데이터로 학습
RLHF (인간 피드백 강화학습)	유용·정직·무해한 답, 사람 선호 정렬	InstructGPT 2022 [25]	개념 소개 (실습은 SFT 단계까지)
Chain-of-Thought (사고 사슬)	다단계 추론	Wei et al. 2022 [26]	프롬프트 기법으로 소개 — 수학추론 (GSM8K-ko) 태스크와 이어짐
Test-time Compute (추론 시 연산)	어려운 문제에 더 오래 생각	Self-Consistency·ToT·ReAct [27-29], o1	개념 소개
초장문 처리	긴 문맥의 메모리·속도, 문맥 창 확장	FlashAttention [30], Position Interpolation [31]	<code>max_length</code> 설정에서 체감
LoRA / QLoRA	거대 모델을 작은 자원으로 미세조정	Hu et al. 2022 [32], Dettmers et al. 2023 [33]	★ 실습의 핵심 — 파라미터 약 1%만 학습
MoE (전문가 혼합)	파라미터는 크게, 연산은 작게	Shazeer 2017 [34], Survey [35]	최신 모델 구조 이해용

9.1 RAG — 검색 증강 생성 (Retrieval-Augmented Generation)

◆ 문제: 모델은 학습 시점 이후를 모르고, 모르는 것도 그럴듯하게 말한다 (환각)

◆ 해결: 답하기 전에 외부 지식을 검색해서 프롬프트에 넣어 준다

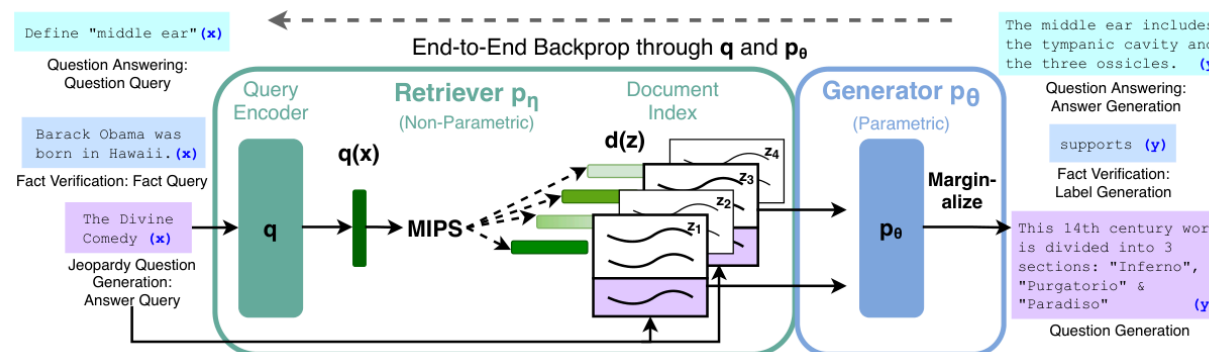
- ① 질문을 벡터로 바꿔 문서 저장소(벡터 DB)에서 관련 문서 k개 검색 (Retriever)
- ② 검색된 문서 + 질문을 생성 모델에 입력 (Generator) → 근거에 기반한 답과 출처

◆ 장점

- 최신 정보·사내 문서 반영, 출처 제시, 모델 재학습 없이 지식 갱신 — 기업 도입의 표준 패턴

◆ 미세조정과의 관계 (자주 받는 질문)

- 지식을 넣는 데는 RAG, 형식·말투·태스크 수행 방식을 가르치는 데는 미세조정 — 둘은 보완 관계
- 내일 실습은 '방식'을 가르치는 쪽(미세조정). '지식'이 필요하다면 그 위에 RAG를 얹는다



RAG 개요 (Figure 1) · 출처: [23] Lewis et al., 2020

9.2 Instruction Tuning — 지시문으로 가르치기 (★ 실습 핵심)

◆ FLAN (Google, 2022): 다양한 태스크를 자연어 지시문 형태로 바꿔 미세조정

- 60여 개 NLP 데이터셋을 "다음 문장을 스페인어로 번역하세요: ..."처럼 지시문+입력+답 형식으로 변환
- 그 결과, **학습에 없던 새 태스크**의 지시도 따른다(zero-shot) — 137B FLAN이 175B GPT-3 few-shot을 앞섬 (그림 하단)

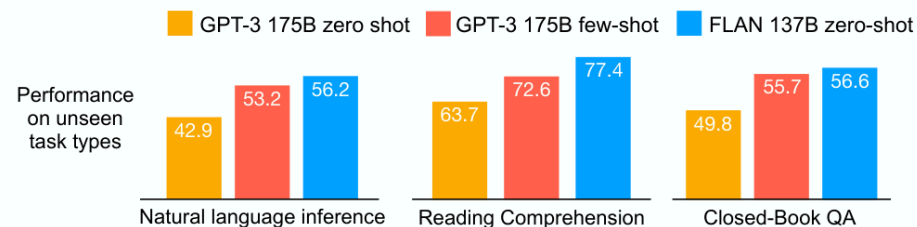
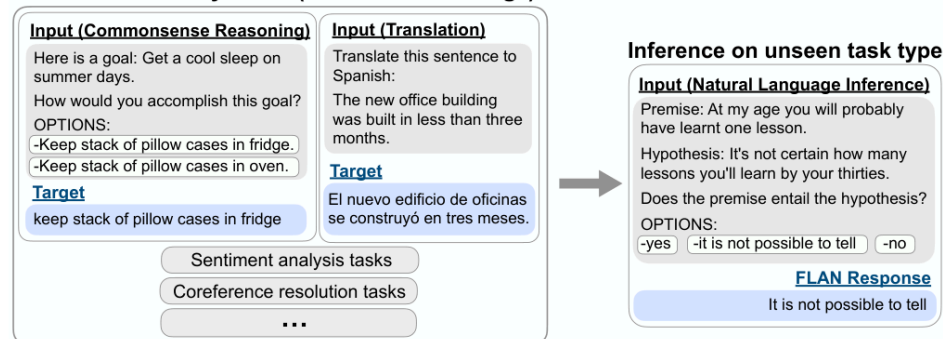
◆ 왜 되는가

- 모델은 이미 사전학습으로 능력을 갖고 있다. 지시 미세조정은 능력을 '새로 넣는' 것이 아니라 '지시에 반응하는 인터페이스'를 학습시킨다
- 그래서 데이터가 많지 않아도 된다 — 수천~수만 건, 다양성이 양보다 중요

◆ 내일 실습이 정확히 이것이다

- 6개 태스크 × 600~1,000건 = **4,800건**의 지시문 데이터 (주제분류·문장유사도·개체명인식·기계독해·수학추론·SQL생성)
- 한 모델이 지시문만 보고 여섯 가지 일을 한다 — BERT·T5처럼 태스크마다 모델을 따로 만들지 않는다

Finetune on many tasks ("instruction-tuning")



FLAN: 지시 미세조정과 미학습 태스크 성능
(Figure 1, 5) · 출처: [24] Wei et al., 2022

9.3 RLHF — 인간 피드백으로 '태도'를 가르치다

◆ InstructGPT (OpenAI, 2022): ChatGPT의 직접적인 전신

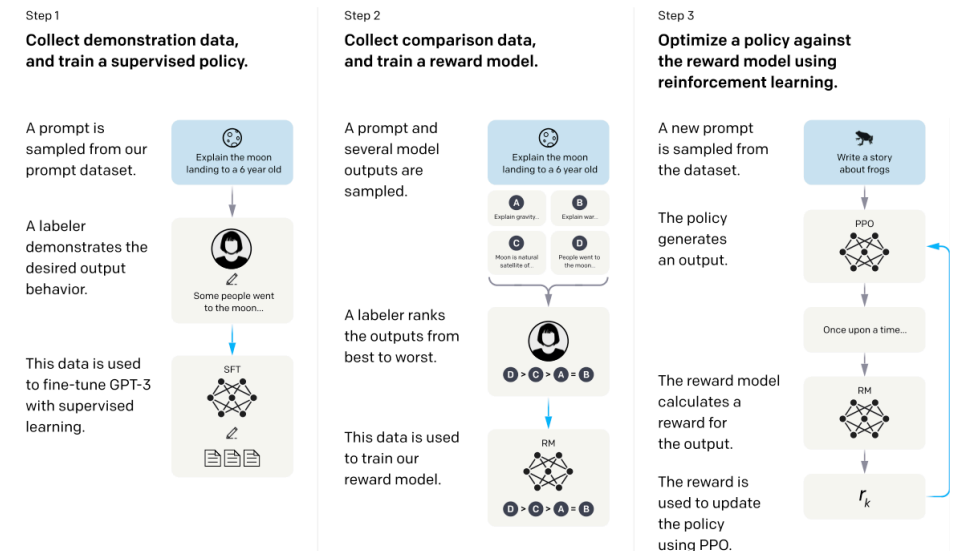
- 1단계 SFT: 사람이 쓴 시범 답변으로 지도 미세조정 (← 내일 실습이 여기까지)
- 2단계 보상 모델: 한 질문에 대한 여러 답을 사람이 순위 매김 → '사람이 좋아하는 답'을 점수화하는 모델 학습
- 3단계 강화학습(PPO): 보상 모델 점수가 높아지도록 언어모델을 갱신

◆ 결과

- 1.3B InstructGPT의 답을 사람들이 175B GPT-3의 답보다 선호 — 규모보다 정렬 (alignment)이 체감 품질을 결정
- 유용성(helpful)·정직성(honest)·무해성(harmless)의 '3H'

◆ 이후의 진화

- DPO(2023): 보상 모델·강화학습 없이 선호 쌍으로 직접 최적화 — 더 간단·안정. 오픈 모델 대부분이 채택
- RLAI / 헌법적 AI(Anthropic): 사람 대신 AI가 원칙에 따라 피드백
- 추론 모델의 RL(o1, DeepSeek-R1): 정답 검증이 가능한 문제(수학·코드)로 보상 → 사고 사슬 학습



9.4 Chain-of-Thought — '단계별로 생각해 보자'

◆ 관찰: 큰 모델에게 풀이 과정을 예시로 보여 주면 다단계 추론 정확도가 급상승 (Wei et al. 2022)

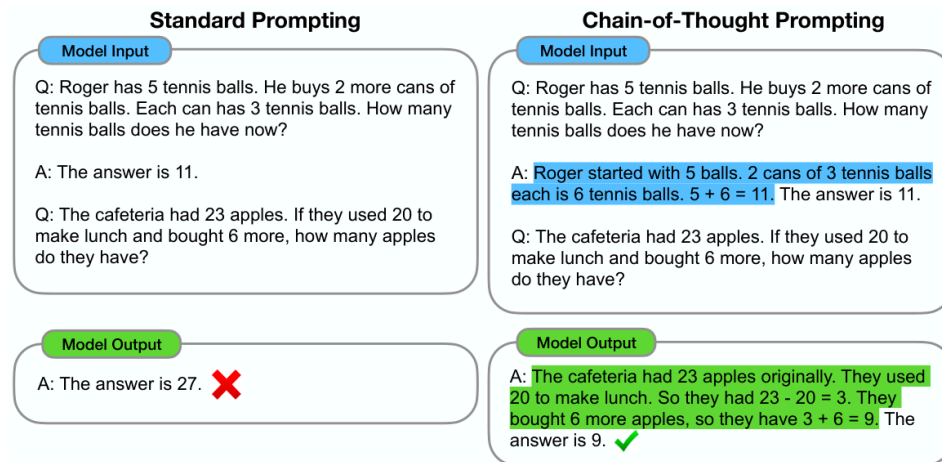
- 표준 프롬프트: 문제 → 답 (그림 왼쪽, 오답)
- CoT 프롬프트: 문제 → 풀이 과정 → 답 (그림 오른쪽, 정답). 예시 몇 개에 풀이를 적어 주기만 하면 된다
- Zero-shot CoT: 예시 없이 "Let's think step by step" 한 문장만으로도 효과 (Kojima et al. 2022)

◆ 왜 되는가

- 중간 단계를 토큰으로 써 내려가면 각 단계가 다음 단계의 문맥이 된다 — 계산을 '외부 메모 (scratchpad)'에 나눠 하는 효과
- 창발 능력의 대표 사례: 약 100B 이상에서 효과가 나타나고 작은 모델은 오히려 나빠질 수 있음

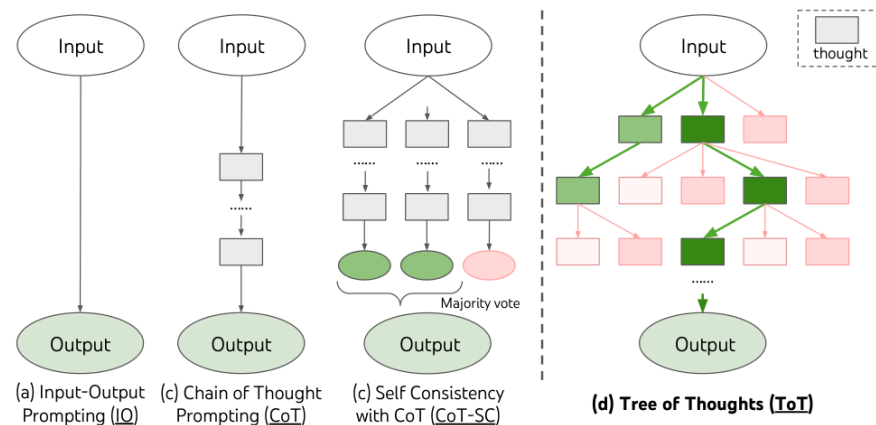
◆ 추론 모델로의 연결

- 프롬프트로 유도하던 CoT를 강화학습으로 모델 안에 내재화한 것이 o1 계열 — '생각 중' 표시의 정체



9.5 Test-time Compute — 답할 때 더 많이 계산하기

- ◆ 아이디어: 학습을 더 하지 않고, 추론 시점에 연산을 더 써서 성능을 올린다
- ◆ Self-Consistency (2023) [28]
 - 같은 문제를 여러 번(예: 40번) 풀게 해서 다수결 — 하나의 사고 사슬보다 훨씬 안정적
- ◆ Tree of Thoughts (2023) [29]
 - 여러 갈래의 생각을 트리로 펼치고, 각 갈래를 평가해 가망 없는 쪽은 버린다 — 탐색 (search)을 도입
- ◆ ReAct (2023) [27]
 - '생각(Reason) → 행동(Act: 검색·계산기 호출) → 관찰' 반복 — 도구 사용과 추론의 결합, 에이전트의 원형
- ◆ 추론 모델 (o1, 2024~)
 - 위 아이디어들을 강화학습으로 모델 내부에 학습 — 사고 길이를 늘리면 성능이 오르는 새 스케일링 축
 - 비용: 같은 질문에 10~100배의 토큰. 그래서 '빠른 모드/생각 모드'를 고르는 하이브리드가 표준이 됨



9.6 초장문 처리 — 문맥 창을 어떻게 100만 토큰까지 늘렸나

◆ 문제 1: Self-Attention은 길이의 제곱(n^2)으로 메모리·시간이 늘어난다

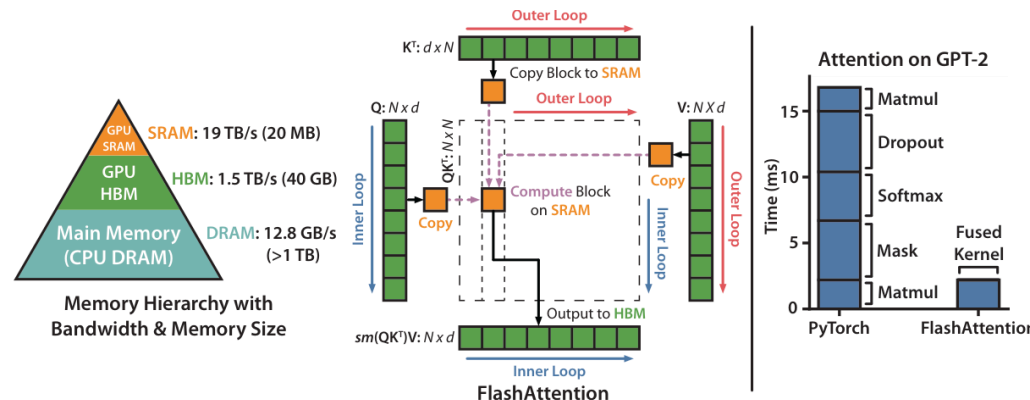
- FlashAttention (2022) [30]: 계산 순서를 GPU 메모리 계층(SRAM/HBM)에 맞게 재배치 — 정확도 손실 없이 2~4배 빠르고 메모리는 선형
- 이후 FlashAttention-2/3, 그리고 PagedAttention(vLLM) 등 추론 최적화가 뒤따름

◆ 문제 2: 학습 때 본 길이(예: 4K)보다 긴 입력에서 위치 정보가 깨진다

- Position Interpolation (2023) [31]: RoPE 위치를 '늘리는' 대신 '압축'해 학습 범위 안으로 — 소량 미세조정으로 4K → 32K
- 이후 YaRN, 그리고 처음부터 긴 문맥으로 학습 — Gemini 1.5의 100만 토큰, Llama 4 Scout의 1,000만 토큰

◆ 실무에서 알아둘 것

- 문맥 창이 크다고 다 잘 읽는 것은 아니다 — '건초더미 속 바늘(needle-in-a-haystack)' 평가로 확인
- 내일 실습의 `max_length=1024`: 짧게 잡으면 빠르고 메모리 절약, 긴 지문(독해)은 잘릴 수 있어 절충



GPU 메모리 계층과 FlashAttention (Figure 1) · 출처: [30] Dao et al., 2022

9.7 LoRA — 파라미터 1%만 학습하기 (★ 실습 핵심)

◆ 문제: 전체 미세조정(Full Fine-tuning)은 모델 전체 가중치를 갱신

- 7B 모델이면 가중치 14GB + 옵티마이저 상태 등으로 GPU 메모리 100GB 이상, 태스크마다 14GB 사본

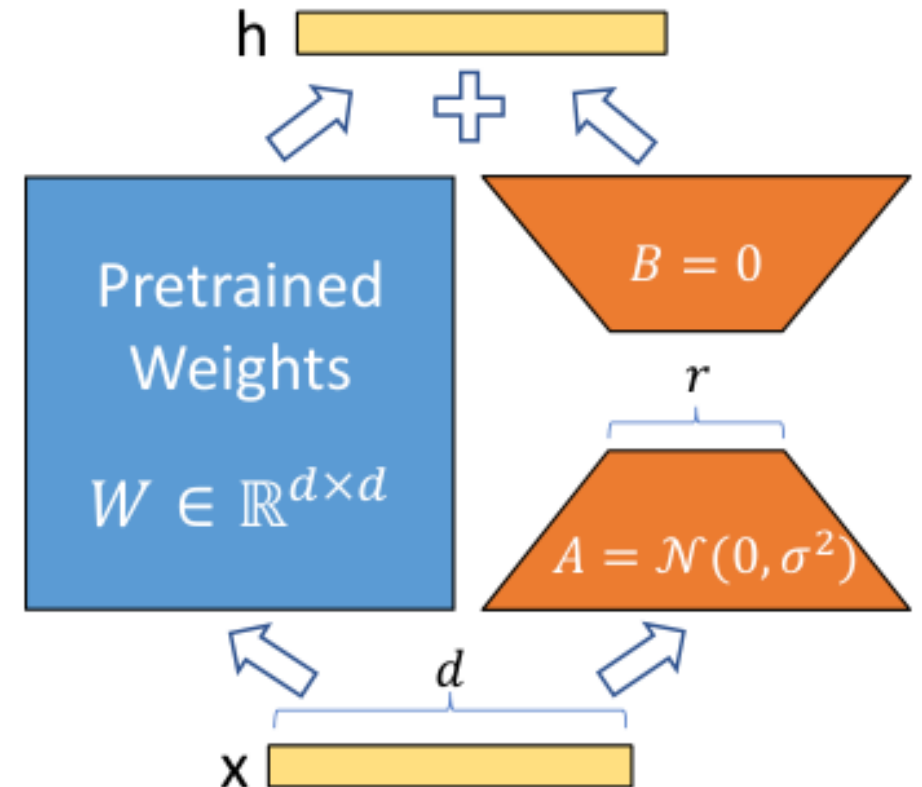
◆ LoRA (Low-Rank Adaptation, 2022) [32]

- 원래 가중치 $W(d \times d)$ 는 그대로 얼려 두고, 옆에 작은 두 행렬 $A(d \times r) \cdot B(r \times d)$ 를 붙여 그 둘만 학습 ($r = 8 \sim 64$)
- 출력 = $W \cdot x + B \cdot A \cdot x$. 학습이 끝나면 $W + BA$ 로 합칠 수 있어 추론 속도 손실 없음
- 학습 파라미터가 0.1~1%로 줄어 메모리·시간 절감, 어댑터 파일은 수십 MB — 태스크마다 어댑터만 교체

◆ QLoRA (2023) [33]

- 얼린 가중치를 4비트로 양자화해 메모리를 다시 1/4로 — 65B 모델을 GPU 한 장(48GB)에서 미세조정
- 성능은 16비트 전체 미세조정과 거의 동일 — 소비자 GPU·노트북에서의 LLM 미세조정을 현실화

- ◆ 💡 직관: 미세조정으로 바뀌는 부분은 '저차원'이다 — 모델은 이미 거의 다 알고 있고, 방향만 조금 틀어 준다



9.7 LoRA, 실제 숫자로 — 내일 학습할 모델에서 실측

모델	전체 파라미터	LoRA 학습 파라미터 (r=16)	비율	학습 시간 (1 epoch)	GPU 피크 메모리
A.X-4.0-Light (SKT)	7,299,995,136	40,370,176	0.55%	20분	17.1 GB
EXAONE-4.0-1.2B (LG)	1,294,628,608	15,237,120	1.18%	10분	4.3 GB
kanana-1.5-2.1B-base (카카오)	2,109,982,464	23,003,136	1.09%	15분	6.1 GB
Qwen3.5-2B (Alibaba)	1,898,644,288	16,819,200	0.89%	35분	7.1 GB
gpt-oss-20B (OpenAI, MoE)	20,940,000,000	30,081,024	0.14%	48분	42.9 GB
(비교) <code>klue/roberta-base</code> 전체 학습	약 110,000,000	110,000,000	100%	태스크당 수 분	수 GB
(비교) <code>paust/pko-t5-base</code> 전체 학습	276,000,000	전체 (100%)	100%	18분	약 8 GB
(비교) 7B 모델 전체 미세조정	7,000,000,000	7,000,000,000	100%	—	100 GB 이상

질문에 대한 답: "1%만 바뀌서 될까?" → 다음 '실습 연결' 절에서 학습 전/후 성능 변화를 실측으로 보여 드립니다.

9.8 MoE — 전문가 혼합 (Mixture of Experts)

◆ 아이디어: 파라미터는 아주 크게, 토큰마다 실제로 쓰는 연산은 작게

- Feed-Forward 층을 여러 '전문가(Expert)' 네트워크로 복제하고, 라우터(Gating)가 토큰마다 상위 k개(보통 1~2개)만 활성화
- 예) 총 671B 파라미터, 토큰당 활성 37B (DeepSeek-V3) — 저장은 크지만 계산은 37B 모델 수준

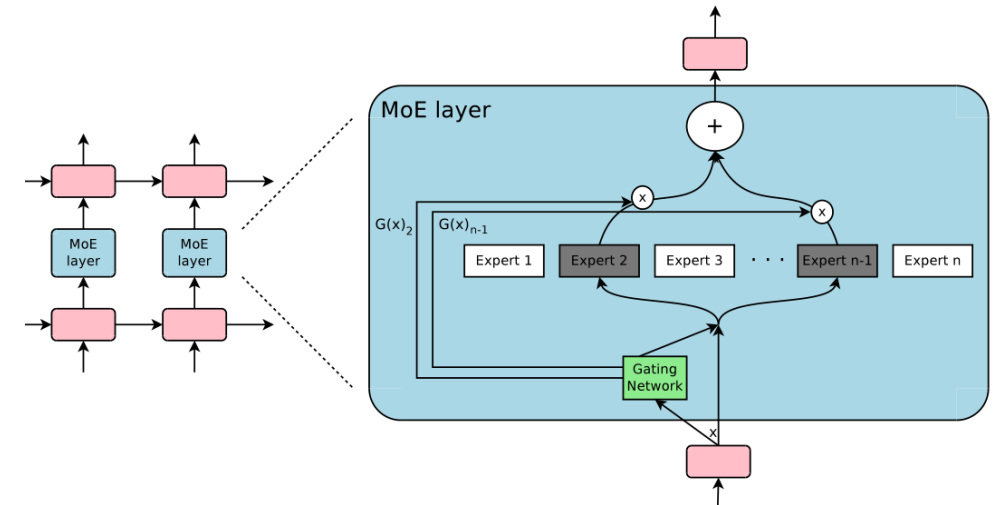
◆ 역사

- Shazeer et al. 2017 [34]: 희소 게이트 MoE로 1,370억 파라미터 LSTM (당시 최대)
- Switch Transformer(2021) → Mixtral 8x7B(2023, 오픈) → GPT-4(추정), Gemini 3, DeepSeek-V3/V4, Llama 4, Qwen3, Gemma 4 26B-A4B

◆ 장단점

- 장점: 같은 연산 예산에서 더 높은 성능, 학습·추론 비용 절감
- 단점: 메모리(모든 전문가를 올려야 함), 학습 불안정(전문가 불균형), 미세조정이 까다로움

◆ 소형 실습 모델은 대부분 dense(비MoE) — MoE는 대형 최신 모델 구조를 읽을 때 필요한 개념



실습 연결 BERT · T5 · GPT 계열 세 방식의 차이 — 2일차 예고

- 1 같은 데이터, 같은 평가셋, 같은 지표로 세 방식(인코더 · 인코더-디코더 · 디코더)을 정면 비교한다
- 2 구현 방식의 차이: 태스크별 head · text-to-text · 지시문 하나
- 3 성능의 차이, 그리고 인코더로는 풀 수 없는 태스크 — 수학추론 · SQL생성
- 4 2일차 실습 로드맵과 준비물

T5 기준선 — paust/pko-t5-base로 같은 네 태스크를 푼다

◆ 이번 과정의 세 번째 방식 — 인코더-디코더

- BERT처럼 태스크마다 head를 새로 설계하지 않는다. 입력 텍스트와 정답 텍스트만 정하면 된다
- 다만 태스크마다 따로 학습한다 — 모델 하나로 여섯 가지를 하는 것은 GPT 계열 쪽 이야기다

◆ 태스크별 입력 → 정답 (전부 텍스트로 통일)

- 주제분류: 뉴스 제목 → 정치 / 문장유사도: 문장 두 개 → 3.8
- 개체명인식: 문장 → 개체 목록 텍스트 / 기계독해: 지문 + 질문 → 정답 문자열
- 수학추론·SQL생성: 문제 → 풀이와 최종 답 / SQL 한 줄 — 인코더로는 못 하지만 T5는 된다

◆ 무엇을 보려고 넣었나

- 인코더(BERT)와 디코더(GPT 계열) 사이에 있는 구조가 같은 태스크에서 어디쯤 오는지 — 세 방식 비교의 가운데 축
- 성능은 같은 학습셋에서 네 태스크 평균 72.6 (전체 학습 시 83.8) — 다음 두 슬라이드의 표에 같은 300문항, 같은 채점으로 들어간다

세 방식 비교 — 구현 관점

항목	BERT 인코더 (klue/roberta-base, 1.1억)	T5 인코더·디코더 (paust/pko-t5-base)	GPT 계열 디코더 (skt/A.X-4.0-Light 7.3B 등)
모델 구조	인코더 + 태스크별 출력층(head) — 분류·회귀·토큰분류·스팬추출이 각각 다른 코드	인코더-디코더 — 모든 태스크를 입력 텍스트 → 출력 텍스트로 통일	디코더 하나 — 모든 태스크가 '지시문 → 텍스트 생성'
다루는 태스크	주제분류·문장유사도·개체명인식·기계독해 네 개	위 네 개 + 수학추론·SQL생성	여섯 개 전부
학습 방식	전체 파라미터 갱신, 태스크마다 별도 학습	전체 파라미터 갱신, 태스크마다 별도 학습	LoRA 어댑터 약 0.5~1%만 갱신, 여섯 태스크를 한 번에 학습 (TRL SFTTrainer)
데이터 형식	태스크별 라벨 (정수 클래스, 실수 점수, BIO 태그, 시작/끝 위치)	입력 텍스트 → 정답 텍스트	지시문 + 입력 + 답(텍스트/JSON)의 대화 형식 — 사람이 읽을 수 있음
산출물	태스크별 모델 파일 네 개 (각 ~450MB)	태스크별 모델 파일 네~여섯 개	베이스 모델 1개 + 어댑터 1개 (~60MB), 태스크 추가 = 데이터 추가
출력 형식	확률·점수·위치 → 형식이 깨질 수 없음	자유 텍스트 → 형식 오류 가능	자유 텍스트 → 형식 오류 가능 → <code>broken_format</code> 비율을 함께 측정
학습 전 성능	무작위 head → 학습 전에는 아무것도 못 함	전체 미세조정을 전제로 하므로 학습 전 성능은 따로 보지 않는다	지시문만으로도 일부 수행(zero-shot) — 단, 작은 모델은 약하다
새 태스크 추가	새 head 설계 + 코드 수정 + 재학습	입력·출력 텍스트 형식만 정하면 된다	지시문 예시 몇백 건 추가 후 재학습 — 코드 변경 없음

Hugging Face — 모델을 어디서 가져오나

◆ Hub — 모델·데이터셋·Spaces 가 올라오는 공개 저장소

- 우리 코드가 아는 것은 **저장소 주소 하나**뿐입니다 — `from_pretrained("skt/A.X-4.0-Light")`
- 처음 한 번 내려받아 `~/.cache/huggingface/hub` 에 둡니다. 두 번째부터는 받지 않습니다

◆ 모델 카드 — 저장소 첫 화면(README)

- 라이선스 · 언어 · 크기 · 사용법이 모두 여기 있습니다. **모델을 쓰기 전에 여기부터 읽습니다**
- 다음 장에서 오늘 학습할 모델의 카드를 함께 읽어 봅니다

◆ gated 저장소와 토큰

- **gated** — 약관에 동의해야 받아지는 저장소가 있습니다. HyperCLOVA X SEED 1.5B 가 그래서 오늘 비교표에서 빠졌습니다
- **토큰** — gated 모델이나 비공개 저장소에 필요합니다. 설정 → Access Tokens 에서 **읽기 전용**으로 만들고 `hf auth login` 으로 한 번 저장합니다
- **⚠ 토큰을 코드나 노트북에 적지 마십시오.** 저장소에 올라가면 남이 내 계정으로 씁니다 — `hf auth login` 이나 환경변수 `HF_TOKEN` 을 씁니다

◆ 오늘 쓰는 모델은 전부 공개입니다 — 토큰을 만들지 않아도 됩니다

A.X-4.0-Light — 오늘 학습할 모델의 카드 읽기

◆ 먼저 — 쓸 수 있는가

- 라이선스 **Apache-2.0** (상용 포함 자유) · 접근 조건 **공개**(gated 아님) · 언어 [ko](#) · [en](#)
- 만든 곳 **SKT AI Model Lab**, 2025년 7월 — 누가 책임지는 모델인지가 카드에 적혀 있습니다

◆ 그다음 — 내 GPU 에 올라가는가, 무엇 위에 지었는가

- 크기 **7.26B**(카드 표기는 7B) · bf16 가중치 약 15GB · 문맥 길이 **16,384 토큰** — 실습은 1,024면 충분합니다
- 베이스 모델 **Qwen2.5** 에 **한국어를 추가학습** · 사용 예시가 [apply_chat_template](#) 이니 그 형식으로 넣으라는 뜻입니다

◆ 마지막 — 무엇을 잘한다고 주장하는가

- 카드의 경량 모델 표: KMMLU **64.15** · CLiCK **68.05** · MMLU **75.43** — 같은 크기대(Qwen3-8B · Qwen2.5-7B · EXAONE-3.5-7.8B · Kanana-1.5-8B)에서 KMMLU·CLiCK 가 가장 높습니다
- ⚠️ 카드 앞머리의 KMMLU 78.3 · GPT-4o 비교는 **72B 짜리 A.X 4.0** 값입니다. 우리가 쓰는 것은 **Light** 이고 표가 따로 있습니다

◆ 카드의 벤치마크는 만든 쪽이 고른 평가입니다 — 오늘 우리가 하는 일은 내 태스크로 직접 재 보는 것이고, 그래서 실습은 학습 전 점수부터 잹니다



실측 미리보기 — 73억 모델, LoRA 1 epoch 전과 후

◆ skt/A.X-4.0-Light (7.3B) · 학습 데이터 4,800건 · 공식

검증셋에서 각 300문항 · 학습셋과 평가셋 겹침 0건

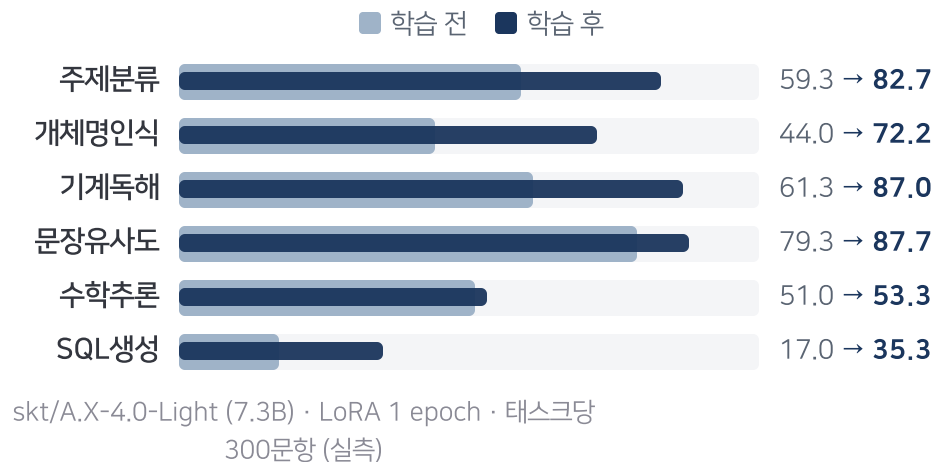
- 주제분류 정확도 **59.3 → 82.7** / 문장유사도 Pearson **79.3 → 87.7** / 개체명인식 F1 **44.0 → 72.2**
- 기계독해 EM **61.3 → 87.0** / 수학추론 EM **51.0 → 53.3** / SQL생성 EM **17.0 → 35.3** · 여섯 태스크 평균 **52.0 → 69.7**

◆ 읽는 법

- 학습 전에 이미 되는 태스크(문장유사도·기계독해)와 학습 후에야 제대로 되는 태스크(SQL생성·개체명인식)를 나눠서 본다 — 출발점이 낮을수록 많이 오른다
- 파라미터 0.55%만 1 epoch 학습해서 얼마나 달라지는지를 본다 — '능력을 새로 넣는 것이 아니라 방식을 가르친다'는 말의 뜻

◆ 2일차에 보게 될 것

- 여러 베이스 모델(한국어 특화 vs 다국어, Base vs Instruct)의 같은 표 + BERT·T5 기준선과의 비교 표·차트



왜 수학추론만 떨어질까 — 형식이 아니라 답이 흔들린다

측정 항목	학습 전	학습 후
정답률 (EM)	47.0	31.0
풀이 길이 중앙값	160자	141자
#### 답 형식을 지킨 건수	82 / 100	90 / 100

흔한 설명은 틀렸다 — 형식이 뒀인 것이 아니다

- "짧은 풀이 형식이 원래의 긴 풀이 습관을 뒀는다"가 흔한 설명이다. 같은 모델로 **직접 재 보니 아니었다**
- 형식은 오히려 **좋아졌고**(82 → 90건) 풀이 길이도 거의 그대로다(160 → 141자). 떨어진 것은 **정답률 하나뿐**이다

같은 문제, 학습 전과 후

- **학습 전** — "240분을 7일로 나누면 매일 약 34분 48분" → #### 34분 48. 답에 근접했지만 **형식이 지저분해 오답 처리**
- **학습 후** — "3주면 $3 \times 7 = 21$ 일, $240 / 21 = 11.428$ " → #### 11.428. 형식은 깨끗한데 **문제를 잘못 읽었다**

그래서 무엇인가

- 여섯 태스크를 한 번에 배우는 과정에서 **계산·추론의 신뢰도가 흔들린다**. 나머지 다섯은 짧고 결정적인 출력을 요구하는데, 그 압력이 여러 단계를 밟아야 하는 수학추론에는 불리하게 작용한다
- **형식을 배우면 이기는 태스크**(주제분류·문장유사도·개체명인식·기계독해·SQL 생성)와 **원래 갖고 있던 추론을 써야 하는 태스크**(수학추론)가 갈린다
- **이번 개편이 그 예다**. 강의장 GPU에서 학습되는 모델을 여럿 재 보고, 파인튜닝 뒤에도 수학추론이 떨어지지 않는 **skt/A.X-4.0-Light**(7.3B)로 수업 모델을 바꿨다 — 같은 데이터·같은 설정인데 EXAONE-4.0-1.2B 는 **44.7 → 32.0**, A.X 는 **51.0 → 53.3**. 데이터를 고치지 않고 모델을 고른 것만으로 달라진다
- 그래서 성능표는 **평균 한 줄로 읽지 않는다** — 오른 칸과 내린 칸을 같이 본다

실측 미리보기 — 세 방식 정면 비교 (같은 300문항, 같은 채점)

◆ 세 방식이 모두 가능한 네 태스크 평균 (주제분류 · 문장유사도 · 개체명인식 · 기계독해)

- BERT 인코더 `klue/roberta-base` — 같은 학습셋 **76.2** / 공식 train 전체 **86.6**
- T5 인코더-디코더 `paust/pko-t5-base` — 같은 학습셋 **72.6** / 전체 **83.8**
- GPT 계열 디코더 + LoRA (통합 4,800건 1 epoch) — `skt/A.X-4.0-Light` **82.4**, 가장 높은 모델 **84.8** (Qwen3.5-4B)

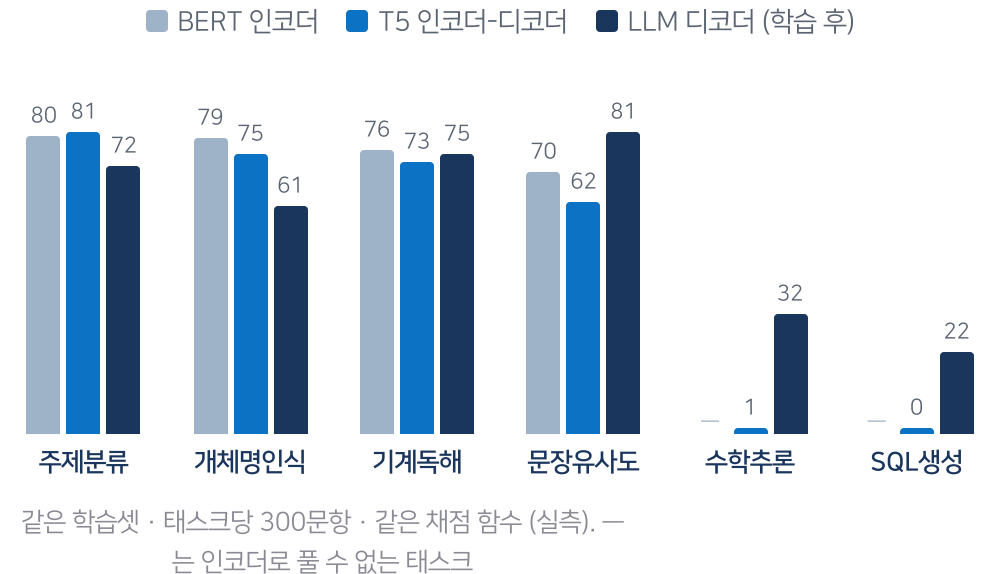
◆ 생성 모델만 가능한 두 태스크 (수학추론 GSM8K-ko · SQL생성 Spider-ko)

- 정답이 클래스 라벨도 지문 속 스펠도 아니라서 BERT 인코더로는 풀 수 없다 — T5와 GPT 계열만 비교 대상이 된다

◆ 세 가지 읽기

- ① 라벨 데이터가 적을 때와 충분할 때 순위가 뒤집히는지 본다
- ② 태스크 성격(분류 · 회귀 · 토큰 라벨 · 스펠 · 생성)에 따라 유리한 구조가 다르다
- ③ 비용: 추론 300건에 BERT 0.1초 vs 생성 모델 수 초~수 분 · 학습 파라미터는 BERT 1.1억 전체 vs LoRA 1천만~5천만

◆ 🧭 결론은 승패가 아니라 조건표 — 라벨 데이터 양 · 출력의 고정성 · 지연 시간 예산 · 태스크 변경 빈도로 고른다



2일차 실습 로드맵 — 내가 학습시킨 모델에게 질문하기까지

- ◆ **0. 준비 (강사가 미리):** 베이스 모델·데이터셋 다운로드, 환경 점검 — 여러분은 저장소를 받고 실행만
- ◆ **1. 데이터 만들기** `build_dataset.py`: 6개 공개 데이터셋 — 주제분류 KLUE-YNAT · 문장유사도 KLUE-STs · 개체명인식 KLUE-NER · 기계독해 KorQuAD · 수학추론 GSM8K-ko · SQL생성 Spider-ko(보너스) → 지시문 대화 형식 JSONL. 학습셋은 train 분할에서만, 평가셋은 validation 분할
- ◆ **2. 학습 전 평가** `evaluate.py`: 베이스 모델 그대로 6태스크 측정 — 오늘 표의 '전' 열을 각자 재현
- ◆ **3. LoRA 학습** `train.py`: TRL SFTTrainer + PEFT, 1 epoch. 학습 중 loss 곡선을 보며 개념 복습 (10~30분)
- ◆ **4. 학습 후 평가:** 같은 문항, 같은 지표로 '후' 열 — 형식 오류율도 확인
- ◆ **5. 데모 페이지** `serve.py`: Flask 웹에서 태스크를 고르고 자유 입력 → 어댑터 컴/꿈으로 학습 전/후 답 비교
- ◆ **6. BERT·T5와 비교:** 1일차 오후의 BERT 인코더·T5 인코더-디코더 결과와 나란히 — 성능·시간·메모리·구현 난이도 표로 정리, 토론
- ◆ **노트북 버전:** 위 단계를 설명과 함께 셀 단위로 나눈 Jupyter 노트북 제공 — 복습·자습용

PART
IV

변화하는 LLM을 대하는 우리의 자세

- 1 LLM의 한계와 위험 — 무엇을 믿고 무엇을 검증할 것인가
- 2 질문하는 방식의 변화 — 프롬프트에서 위임으로
- 3 도구 선택의 기준 — API인가, 오픈웨이트 미세조정인가
- 4 AI 시대의 연구자·개발자에게 남는 일



10. LLM의 한계와 위험 — 성능표가 말해 주지 않는 것

◆ 환각(Hallucination): 그럴듯하지만 틀린 말

- 확률적 생성의 본질적 속성 — 최신 모델도 사실 오류를 0으로 만들지 못함 (GPT-5.4가 5.2 대비 33% 감소를 '성과'로 발표하는 수준)
- 대응: 근거 제시(RAG), 검증 가능한 출력 형식, 사람의 최종 확인 — '검증 비용'을 설계에 넣어야 함

◆ 편향과 안전: 학습 데이터의 편향을 그대로 학습, 유해 요청에 대한 거부와 유용성 사이의 균형

◆ 평가의 함정

- 벤치마크 오염(학습 데이터에 시험 문제가 섞임), 벤치마크 과적합 — 성능표의 숫자는 '그 시험'에서의 숫자
- 같은 이유로 내일 실습은 학습셋(train)과 평가셋(validation)을 엄격히 분리 — 평가셋을 학습에 넣지 않는다

◆ 비용과 의존

- 추론 모델은 토큰을 10~100배 소비, API 요금·지연 시간·데이터 반출 문제 — 특히 공공·의료·금융에서 오픈웨이트 수요의 배경

◆ ⚠️ 작은 모델의 한계도 알아야 한다

- 소형 모델은 지시를 따르되 지식·추론이 얇다 — 내일 데모에서 '형식은 맞는데 내용이 틀린' 답을 보게 될 것. 그것이 정상이다

11. 질문하는 방식의 변화 — 프롬프트에서 위임으로

시기	사람이 하는 일	모델이 하는 일	필요한 역량
2020 GPT-3	예시(few-shot)를 정교하게 골라 붙인다	패턴을 이어서 완성	프롬프트 작성 기술
2022 ChatGPT	자연어로 지시·대화, 결과를 고쳐 달라고 요청	지시를 따르고 대화 맥락 유지	명확한 요구 표현, 결과 검토
2024 추론 모델	문제와 제약 조건을 정확히 기술, 생각 시간을 허용	스스로 단계를 나눠 풀고 검산	문제 정의, 검증 기준 설정
2025~ 에이전트	목표와 완료 조건을 주고, 도구·권한을 허용한다	계획·실행·도구 사용·자기 수정을 수십 단계 반복	위임과 검수, 권한·안전 설계
실습(내일)	지시문 데이터로 '우리 태스크의 방식'을 가르친다	지시문만 보고 6개 태스크 수행	데이터 설계, 평가 설계

12. 도구 선택의 기준 — 상용 API인가, 오픈웨이트 미세조정인가

기준	상용 API (GPT·Gemini·Claude)	오픈웨이트 + 미세조정 (내일 실습 방식)
성능 상한	최고 — 어려운 추론·코딩·장문은 여전히 격차	중소형은 범용 성능 낮음. 단, 좁은 태스크는 미세조정으로 역전 가능
데이터 반출	외부 서버로 전송 (기업용 계약으로 완화 가능)	사내·개인 GPU에서 완결 — 공공·의료·금융의 요구
비용 구조	토큰당 과금, 대량 호출 시 누적. 추론 모델은 10~100배	GPU 고정비. 호출량이 많고 태스크가 고정이면 유리
지연·안정성	네트워크 지연, 모델 버전 변경·단종 위험	고정 버전, 오프라인 가능
맞춤화	프롬프트·RAG·(제한적) 파인튜닝 API	LoRA로 형식·말투·도메인 자유롭게 조정, 어댑터 교체
개발 난이도	낮음 — 호출 몇 줄	환경·GPU·데이터·평가 파이프라인 필요 (내일 그것을 배움)
권고	탐색·프로토타입·최고 성능이 필요할 때	태스크 고정 + 데이터 반출 불가 + 호출량 큼 → 소형 모델 미세조정

13. AI 시대의 연구자·개발자에게 남는 일

◆ 모델이 빨리 좋아진다는 것은, '모델 자체'보다 '모델을 둘러싼 일'의 가치가 커진다는 뜻

◆ 문제 정의와 데이터

- 무엇을 풀 것인지, 어떤 입력과 출력인지, 좋은 답의 기준은 무엇인지 — 지시문 데이터를 만드는 일이 곧 문제 정의

◆ 평가 설계

- 성능표를 읽는 능력에서 성능표를 만드는 능력으로 — 오염 없는 평가셋, 태스크에 맞는 지표, 형식 오류 같은 부수 지표

◆ 도메인 지식과 검증

- 모델의 답이 맞는지 판단할 수 있는 사람이 마지막까지 필요 — 전문성의 가치는 오히려 커진다

◆ 시스템 설계

- RAG·도구·에이전트·미세조정 모델을 어떻게 조합할지, 비용·지연·안전을 어떻게 균형 잡을지

◆ 태도

- 6개월 단위로 갱신되는 지식을 따라가되, 바뀌지 않는 원리(확률적 언어모델, 사전학습-미세조정, 스케일링)를 붙잡는다
- 오늘 자료의 절반은 1년 뒤 낡을 것이고, 나머지 절반은 10년 뒤에도 유효할 것 — 어느 쪽인지 구분하는 눈을 기르자

기술 너머의 성찰 — 왜 하필 '언어'였을까

◆ 오늘 우리가 따라온 길을 한 걸음 물러나서 보면

- 이미지도 소리도 숫자도 아닌 **언어**를 예측하게 했더니, 문법과 사실과 논리와 상식이 함께 따라왔습니다
- 다음 단어를 잘 맞히는 일 하나가 지식·추론·행동으로 이어졌습니다 — 오늘 본 계단 전체가 그 위에 있습니다
- 왜 다른 많은 것 중에 하필 언어였을까요. 이 물음은 기술 안에서 답이 끝나지 않습니다

◆ 언어가 먼저였다는 오래된 기록

- "하나님이 이르시되 빛이 있으라 하시니 빛이 있었고" (창세기 1:3) — **말이 있고, 그 다음에 사물이 있었습니다**
- 강의자 개인의 소회입니다. 언어모델의 원리를 알아 갈수록 이 문장이 더 실감났습니다

◆ 그래서 남는 것은 능력이 아니라 태도입니다

- 세상의 창조와 인류의 질서가 언어를 통해 이루어졌듯, 언어 기술 앞에서도 **선한 의도를 가진 책임 있는 설계자**가 되어야 합니다
- 기술이 확장되는 속도만큼, 그것을 쓰는 사람의 양심과 진실성도 함께 자라야 합니다
- AI가 편하게 해 줄수록 최종 목적지를 정하고 책임을 지는 조타수는 언제나 사람이어야 합니다

오늘의 핵심 정리 — 다섯 문장

- ◆ ① 언어모델은 '다음 토큰의 확률'을 배우는 모델이고, 그것을 크게 키우자 예상치 못한 능력이 나타났다.
 - Transformer(2017) → BERT/GPT(2018) → GPT-3(2020) → 스케일링 법칙과 창발
- ◆ ② 능력은 사전학습에서 오고, 쓸모는 정렬(지시 미세조정·RLHF)에서 온다.
 - 1.3B InstructGPT가 175B GPT-3보다 선호됨 — 2일차 실습의 근거
- ◆ ③ 2024년부터 '추론 시 연산'이 새 스케일링 축이 되었고, 2025~26년은 에이전트의 시대다.
 - o1 → GPT-5.x / Gemini 3.x / Claude 5 — 생각 모드, 도구 사용, 컴퓨터 조작
- ◆ ④ LoRA·QLoRA로 개인 GPU에서 LLM을 미세조정할 수 있고, 좁은 태스크에서는 소형 모델이 대형 범용 모델을 이길 수 있다.
 - 파라미터 약 0.55%만 학습해도 여섯 태스크 평균이 52.0 → 69.7로 오른다 — 73억 모델에서 개체명인식 44.0 → 72.2, SQL 생성 17.0 → 35.3
- ◆ ⑤ 성능표를 읽을 때는 항목·조건·오염·비용을 묻고, 성능표를 만들 때는 평가셋을 학습에 넣지 않는다.
 - 1일차 오후 BERT·T5, 2일차 GPT 계열 — 같은 데이터·같은 평가셋으로 세 방식(인코더 · 인코더-디코더 · 디코더)을 직접 비교한다

이틀의 실습 로드맵 — 1일차 오후부터

시간	내용	여러분이 하게 되는 것	결과물
1일차 오후 ①	환경 점검 · 데이터 살펴보기	저장소 받기, GPU 확인, KLUE(YNAT·STS·NER) 데이터 열어 보기	실행 가능한 환경
1일차 오후 ②	BERT 주제분류 (KLUE-YNAT)	<code>klue/roberta-base</code> 미세조정 → 정확도 측정 → Flask 데모	주제분류 모델
1일차 오후 ③	BERT 문장유사도 · 개체명인식	회귀 head / 토큰분류 head 학습 → Pearson·F1 측정	STS · NER 모델
1일차 오후 ④	T5 인코더-디코더 실습	<code>paust/pko-t5-base</code> 로 같은 세 태스크를 텍스트→텍스트로 미세조정 → 같은 지표로 측정	T5 모델
2일차 오전 ①	기계독해 (BERT 4A · T5 4B)	스팬추출 head 학습 → EM 측정, 같은 문제를 T5로 텍스트 생성으로도	기계독해 모델 2종
2일차 오전 ②	LLM 파인튜닝 개념 복습 · 데이터 만들기	6태스크 지시문 데이터 생성, 학습 전 평가	학습 전 성능표
2일차 오전 ③	LoRA 학습 (1 epoch, 10~30분)	TRL SFTTrainer 실행, 진행률 표에서 손실·토큰 정확도와 10%마다 태스크별 검증 점수 관찰, 하이퍼파라미터 이해	내 어댑터
2일차 오후 ①	학습 후 평가 · 데모 페이지	같은 문항으로 재평가, Flask 데모에서 어댑터 켜/끄 비교	데모 페이지
2일차 오후 ②	BERT · T5 · GPT 계열 세 방식 비교 · 토론	성능·시간·메모리·구현 난이도 표 완성, 어디에 무엇을 쓸지 토론	비교표 · 차트

미션·퀴즈 정답은 노트북 안에서 — 미션 셀 아래 **확인 문제 셀**을 실행하면 보기 네 개가 버튼으로 나옵니다. 하나를 고르면 **정답이면 바로**, 오답이면 **5초 뒤에** 알려 주고, 틀리면 **힌트가 단계별로** 열립니다 (개념 → 참고 자료·검색 링크 → AI에게 이렇게 물어보세요).

[정답 바로 보기] 는 마지막 수단 — "정말 보시겠습니까? 더 생각해 보시겠습니까?" 두 번 확인 후 **10초**를 기다리면 열립니다. 정답 코드는 복사 버튼으로 미션 셀에 붙여 넣습니다. 강의 페이지의 확인문제도 같은 방식입니다.

정답은 이렇게 확인합니다 — 노트북 안의 확인 문제

◆ 보기를 고른다

- 미션 셀 **아래** 확인 문제 셀을 실행하면 보기 네 개가 버튼으로 나옵니다
- 정답이면 **그 자리에서 바로** 해설이 펼쳐지고, 코드는 복사 버튼으로 미션 셀에 붙여 넣습니다

◆ 틀리면 스스로 찾아가게 돕습니다

- 5초 뒤에 그 보기가 지워지고 **왜 아닌지** 한 줄이 뜨며, **힌트가 하나 자동으로** 열립니다
- 힌트는 세 단계 — **개념** → **근거자료·검색 링크** → **AI에게 이렇게 물어보세요** (힌트 버튼도 누르면 5초 뒤에 열립니다)

◆ 그래도 막히면

- **[정답 바로 보기]** — 두 번 확인하고 **10초**를 기다리면 열립니다. 강의 페이지의 확인문제 24개도 같은 카드입니다

미션 m-n-er-1 특수 토큰 자리에는 어떤 라벨을 둘까

개체명인식은 토큰마다 태그를 답합니다. 그런데 토큰 중에는 [CLS] · [SEP] 처럼 원문에 대응하는 글자가 없는 특수 토큰이 섞여 있습니다 (offset 이 (0, 0) 인 자리).

그 자리에는 정답이랄 것이 없으니, 손실 계산에서 **아예 빼야** 합니다. PyTorch 교차엔트로피가 "이 자리는 무시하라"고 알아보는 값이 정해져 있습니다.

```
if e <= s:                # [CLS] · [SEP] 같은 특수 토큰
    labels.append(____)
    prev_type = None
    continue
```

× 0 ⊖ -100

⊖ L.BIO2ID["O"] ⊖ None

힌트 보기 (2/3) 정답 바로 보기

⊖ 는 아닙니다. 0 은 실제 태그 0 의 번호입니다. 무시되지 않고 "여기는 개체가 아니다"를 학습해 버립니다.

힌트 1 · 개념

PyTorch 교차엔트로피에는 특정 라벨 값을 손실에서 제외하는 기능이 있습니다(ignore_index). 그 기본값이 정해져 있고, transformers 의 토큰 분류도 그 값을 그대로 씁니다. 음수입니다.

힌트 2 · 근거자료 · 검색

근거자료: [PyTorch CrossEntropyLoss — ignore_index 기본값](#) · [transformers — 토큰 분류\(라벨 정렬 절\)](#) · 검색: [ignore_index -100 token classification](#)

언어모델은 '다음 단어'를 맞히는 법을 배웠을 뿐인데
우리는 그 위에 지식과 추론과 행동을 얹었습니다.
언어로 세상이 지어졌다는 오래된 기록 앞에서,
내일은 여러분 손으로 그 한 조각을 얹어 봅니다.

참고 자료 (1/3) — 언어모델의 발전과 평가

- [1] A. Vaswani et al., Attention Is All You Need, NeurIPS, 2017. arXiv:1706.03762
- [2] J. Devlin et al., BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, NAACL, 2019. arXiv:1810.04805
- [3] C. Raffel et al., Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5), JMLR, 2020. arXiv:1910.10683
- [4] A. Radford et al., Improving Language Understanding by Generative Pre-Training (GPT-1), OpenAI, 2018.
- [5] A. Radford et al., Language Models are Unsupervised Multitask Learners (GPT-2), OpenAI, 2019.
- [6] T. Brown et al., Language Models are Few-Shot Learners (GPT-3), NeurIPS, 2020. arXiv:2005.14165
- [7] D. Hendrycks et al., Measuring Massive Multitask Language Understanding (MMLU), ICLR, 2021. arXiv:2009.03300
- [8] Y. Wang et al., MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark, NeurIPS, 2024. arXiv:2406.01574
- [9] D. Rein et al., GPQA: A Graduate-Level Google-Proof Q&A Benchmark, COLM, 2024. arXiv:2311.12022
- [10] D. Hendrycks et al., Measuring Mathematical Problem Solving With the MATH Dataset, NeurIPS, 2021. arXiv:2103.03874
- [11] M. Chen et al., Evaluating Large Language Models Trained on Code (HumanEval), 2021. arXiv:2107.03374
- [12] C. Jimenez et al., SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, ICLR, 2024. arXiv:2310.06770
- [13] L. Phan et al., Humanity's Last Exam, Nature, 2026. arXiv:2501.14249
- [14] OpenAI, GPT-4 Technical Report, 2023. arXiv:2303.08774
- [15] OpenAI, Learning to Reason with LLMs (o1), 2024. openai.com/index/learning-to-reason-with-llms
- [16] OpenAI, Introducing OpenAI o3 and o4-mini, 2025.
- [17] OpenAI, Introducing GPT-5.2, 2025. [18] OpenAI, Introducing GPT-5.3-Codex, 2026.
- [19] Wikipedia, GPT Image (en.wikipedia.org/wiki/GPT_Image), 2026.
- [20] Wikipedia, Gemini (language model), 2026. [21] Wikipedia, Claude (language model), 2026. [22] Wikipedia, Llama (language model), 2026.

참고 자료 (2/3) — 핵심 요소기술과 최신 모델

- [23] P. Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS, 2020. arXiv:2005.11401
- [24] J. Wei et al., Finetuned Language Models Are Zero-Shot Learners (FLAN), ICLR, 2022. arXiv:2109.01652
- [25] L. Ouyang et al., Training Language Models to Follow Instructions with Human Feedback (InstructGPT), NeurIPS, 2022. arXiv:2203.02155
- [26] J. Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, NeurIPS, 2022. arXiv:2201.11903
- [27] S. Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models, ICLR, 2023. arXiv:2210.03629
- [28] X. Wang et al., Self-Consistency Improves Chain of Thought Reasoning in Language Models, ICLR, 2023. arXiv:2203.11171
- [29] S. Yao et al., Tree of Thoughts: Deliberate Problem Solving with Large Language Models, NeurIPS, 2023. arXiv:2305.10601
- [30] T. Dao et al., FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, NeurIPS, 2022. arXiv:2205.14135
- [31] S. Chen et al., Extending Context Window of Large Language Models via Positional Interpolation, 2023. arXiv:2306.15595
- [32] E. Hu et al., LoRA: Low-Rank Adaptation of Large Language Models, ICLR, 2022. arXiv:2106.09685
- [33] T. Dettmers et al., QLoRA: Efficient Finetuning of Quantized LLMs, NeurIPS, 2023. arXiv:2305.14314
- [34] N. Shazeer et al., Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer, ICLR, 2017. arXiv:1701.06538
- [35] W. Cai et al., A Survey on Mixture of Experts in Large Language Models, IEEE TKDE, 2025. arXiv:2407.06204
- [36] J. Kaplan et al., Scaling Laws for Neural Language Models, 2020. arXiv:2001.08361
- [37] J. Wei et al., Emergent Abilities of Large Language Models, TMLR, 2022. arXiv:2206.07682
- [38] OpenAI, Introducing GPT-5, 2025. [39] OpenAI, GPT-5.4 / GPT-5.5 / GPT-5.6 발표, 2026. openai.com/index
- [40] Google DeepMind, Gemini 3 / 3.1 Pro / 3.x Flash 발표, 2025–2026. blog.google · deepmind.google
- [41] Anthropic, Claude 4 / 4.5 / 4.6–4.8 / Fable 5 · Mythos 5 · Sonnet 5 · Opus 5 발표, 2025–2026. anthropic.com/news

참고 자료 (3/3) — 오픈웨이트·한국어·실습

- [42] Google DeepMind, Gemma 4 (E2B/E4B/12B/26B-A4B/31B), 2026. [43] DeepSeek, DeepSeek-V4, 2026. [44] Alibaba Qwen Team, Qwen3.5 / Qwen3.8, 2026.
- [45] 과학기술정보통신부·NIPA, 독자 AI 파운데이션 모델 프로젝트 2차 평가 결과, 2026. 8. 18.
- [46] LG AI Research, EXAONE 4.0: Unified Large Language Models Integrating Non-reasoning and Reasoning Modes, 2025. arXiv:2507.11407
- [47] Kakao, kanana-1.5 (huggingface.co/kakaocorp), 2025. [48] KT, Mi:dm 2.0 (huggingface.co/K-intelligence), 2025. [49] NAVER, HyperCLOVA X SEED, 2025.
- [50] Hugging Face, TRL (huggingface.co/docs/trl) · PEFT (huggingface.co/docs/peft) · Transformers documentation.
- [51] H. Touvron et al., LLaMA: Open and Efficient Foundation Language Models, 2023. arXiv:2302.13971
- [52] R. Schaeffer et al., Are Emergent Abilities of Large Language Models a Mirage?, NeurIPS, 2023. arXiv:2304.15004
- [53] W. Fedus et al., Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity, JMLR, 2022. arXiv:2101.03961
- [54] W. Zhao et al., A Survey of Large Language Models, 2023. arXiv:2303.18223
- [55] S. Park et al., KLUE: Korean Language Understanding Evaluation, NeurIPS Datasets and Benchmarks, 2021. arXiv:2105.09680
- [56] S. Lim et al., KorQuAD1.0: Korean QA Dataset for Machine Reading Comprehension, 2019. arXiv:1909.07005
- [57] 이기창, Do it! BERT와 GPT로 배우는 자연어 처리 (ratsgo nlpbook), 이지스퍼블리싱, 2021. ratsgo.github.io/nlpbook
- [58] 3Blue1Brown, But what is a GPT? / Attention in transformers, visually explained, 2024. youtube.com/@3blue1brown
- [59] 실습 저장소: github.com/chrisjihee/DeepKNLP-26.09 (2일차 LLM 파인튜닝) · github.com/chrisjihee/DeepKNLP-26.03 (1일차 BERT)
- [60] PAUST, pko-t5-base (huggingface.co/paust/pko-t5-base) — 1일차 T5 인코더-디코더 기준선
- [61] K. Cobbe et al., Training Verifiers to Solve Math Word Problems (GSM8K), 2021. arXiv:2110.14168 — GSM8K-ko의 원본
- [62] T. Yu et al., Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL, EMNLP, 2018. arXiv:1809.08887 — Spider-ko의 원본
- [63] 과학기술정보통신부, 독자 AI 파운데이션 모델 프로젝트 정예팀 선정 결과, 2025. 8. 5.
- [64] 과학기술정보통신부, 독자 AI 파운데이션 모델 프로젝트 1차 단계평가 결과, 2026. 1. 15.
- [65] ETRI 언어지능연구실, 공개 모델 (huggingface.co/etri-lirs) — EAGLE · KEByT5 계열. 실측은 실습 저장소 docs/ETRI-MODELS.md

Electronics and Telecommunications Research Institute

THANKS